shingja 的工科生手册
竞赛、项目、工具，和一些踩过的坑

面向北京科技大学工科本科生 · 先讲怎么做，再讲为什么

开始阅读　模板与清单

信息力

检索、甄别、文件、备份、终端。先把信息来源和文件底座弄稳。

文档力

Word、Excel、PPT、Markdown、图示。能交付，才谈得上成果。

代码与数据力

环境、Git、GitHub、数据、可视化。让结果可复现。

项目推进力

选赛、选题、组队、申报、答辩、复盘。只给框架，不给方向。

竞赛名录

北京科技大学 2026-09 版认可竞赛，附工科生优先清单。

经验卷

只讲决策、踩坑和取舍，不写具体项目、奖项。

这本手册怎么用

适用场景

你刚进大学，或者第一次准备竞赛、第一次写申报书、第一次把代码交给别人。网上资料很多，但大多是零散的技巧，缺一条从“不会”到“能交付”的路径。

这本手册补的是这条路径。它不提供竞赛题库，不提供可直接提交的项目方案，也不替你决定做什么方向。它只讲三件事：

1. 工具怎么用，才能少返工。
2. 项目怎么推，才能按时交付。
3. 哪些线不能踩，才能不把成果和信用一起赔进去。

最小做法

先按你现在的阶段选一条路线，不要从头读到尾。

你的情况	建议路线
大一，刚接触大学学习	卷一“信息力” → 卷二“文档力”
第一次参赛，不知道从哪下手	竞赛名录 → 卷四“竞赛值不值得参加” → 对应工具章节
已经是项目负责人	卷四的组队、项目管理、答辩、复盘
准备复盘/交接	卷四的复盘方法

每篇正文统一按六段写：

1. **适用场景**：什么时候该看这篇。
2. **最小做法**：今天就能照着做的步骤。

3. **进阶做法**：有余力再优化。
4. **常见坑**：别人已经踩过的。
5. **自检清单**：做完逐条打勾。
6. **版本与时效**：这篇基于什么版本、什么时候可能过时。

进阶做法

把手册当成“检查表”，不是“教科书”。

- 做项目时，只翻当前阶段对应的两三个章节。
- 做完一个阶段，回到自检清单逐条确认。
- 遇到手册没写的问题，先记录“场景、尝试、结果”，再决定要不要补进手册。

如果你要写自己的经验，建议用“决策—依据—结果—复盘”四段，而不是流水账。读者要的是判断方式，不是你做了多少事。

常见坑

- **从头读到尾**：内容按能力分卷，不是按课程顺序，通读容易失去重点。
- **把工具当目标**：学了一堆软件，但没做过一个完整交付。
- **照搬别人的项目**：方向每年都变，照搬既容易过时，也可能涉及别人的未公开内容。
- **忽略合规**：引用、AI 使用、隐私、版权，任何一项出问题，成果都可能作废。
- **不标注时效**：政策、软件界面、竞赛规则都会变，旧教程可能把人带偏。

自检清单

- [ ] 我知道自己现在该读哪一卷。
- [ ] 我读完一篇后，能说出“最小做法”的步骤。
- [ ] 我把当前项目的下一步写成了一个可执行动作。
- [ ] 我知道哪些内容不能公开、不能照搬。

- [ ] 我记录了所用软件的版本和参考日期。

版本与时效

本手册从 2026 年 10 月开始编写。软件操作以文中标注的版本为准；竞赛政策以学校当年发布的文件为准。发现内容过时，优先相信官方文件，并在 issue 中标注“章节 + 日期 + 变化点”。

卷一·信息力

工科学习里，很多时间不是花在“做”，而是花在“找”和“理”。这一卷解决四个问题：

1. 去哪里找，找到的东西可不可信。
2. 文件怎么命名、怎么放、怎么备份，才不至于三天后找不到。
3. 终端和系统工具怎么用，才能批量处理而不是手工重复。
4. 磁盘、编码、压缩包这些底层小事，怎么不踩坑。

本卷章节

- 检索与来源验证：从关键词到可验证来源。
- 文件系统与命名：目录、命名、路径、快捷方式。
- 备份与同步：3-2-1 原则、OneDrive 冲突、恢复演练。
- 编码、后缀与压缩包：UTF-8/GBK、BOM、换行、压缩格式。
- 终端与批量操作：PowerShell、路径、管道、批量重命名。
- 浏览器插件与脚本风险：权限、油猴脚本、账号隔离。
- 磁盘空间与软件清理：空间排查、卸载、分区扩容。
- 信息甄别：AI 生成内容、营销号、过期政策。

先读哪篇

如果你现在正准备参赛或写申报书，先读检索与来源验证。如果你经常找不到文件，先补“文件系统与命名”。

检索与来源验证

适用场景

你要查一个竞赛政策、找一个软件的正确用法、确认一条技术结论，或者为申报书找支撑材料。搜索框能给你很多答案，但不能告诉你哪个是真的。

最小做法

第一步：先定义你要找什么

同一个关键词，可能对应四种完全不同的需求：

需求	典型问题	优先来源
事实	报名截止是哪天	官方通知、学校文件
方法	Word 目录怎么自动更新	官方文档、可信教程
数据	某行业近五年产量	统计年鉴、论文、官方数据库
政策	这个竞赛保研认不认	学校/学院当年文件

先写下一句话：“我要找的是____，用于____。”写不出来，说明还没想清楚。

第二步：用“原始出处”反查

看到一个说法，先找它最早来自哪里：

- 政策类：学校教务处、学院官网、竞赛官网的通知原文。
- 技术类：软件官方文档、项目仓库的 README/Release。
- 学术类：论文原文、DOI、出版社页面。

- 数据类：统计年鉴、政府/机构数据库。

二手文章可以帮你入门，但不能作为最终依据。

第三步：看日期和版本

同一篇教程，2022 年的界面和 2026 年的界面可能完全不同。至少确认：

- 发布日期或最后更新日期。
- 适用的软件版本。
- 政策适用的学年/年份。

第四步：交叉验证

关键结论至少找两个独立来源。注意“独立”：

- 两篇互相转载的文章，不算两个来源。
- 都引用同一份二手材料的文章，不算两个来源。
- 官方文件 + 一篇独立解读，才算。

进阶做法

学术检索

1. 把中文关键词翻成英文，再搜一次。
2. 用同义词扩展：`边缘计算` 也搜 `edge computing`、`edge intelligence`。
3. 先找综述，建立地图；再找近三年的原始论文。
4. 用“引文追踪”：找到一篇关键论文，看它引用了谁、又被谁引用。
5. 记录检索式、数据库、日期，方便复现。

给搜索结果做“来源分级”

级别	来源	使用方式
A	官方文件、标准、论文原文	可直接作为依据
B	官方文档、机构教程、教科书	可作方法依据，注意版本
C	个人博客、论坛回答、视频教程	只作线索，需回到 A/B 验证
D	营销号、AI 摘要、无署名搬运	不作为依据

AI 检索的三个坑

- **编造引用**：AI 可能给出不存在的论文、作者、DOI。必须逐条查原文。
- **过期信息**：模型训练有截止时间，政策类问题尤其危险。
- **把总结当原文**：AI 的概括可能漏掉限定条件。关键结论回到原文核对。

AI 适合帮你扩展关键词、解释概念、整理笔记；不适合替你确认事实。

常见坑

- 只搜中文，错过英文一手资料。
- 看到“某乎/某书/某站”的高赞回答就直接用。
- 把培训机构的宣传页当成学校政策。
- 不记录来源，写申报书时找不到出处。
- 用 AI 生成的参考文献充数，答辩时被问住。

自检清单

- [ ] 我能用一句话说清要查什么、用于什么。
- [ ] 我找到了至少一个 A 级来源。
- [ ] 我确认了来源的发布日期/适用版本。

- [ ] 关键结论有两个独立来源。
- [ ] 我保存了链接、标题、作者、日期。
- [ ] 我没有把 AI 摘要当成原文。

版本与时效

本文的方法不依赖具体软件版本。涉及学校政策、竞赛规则时，以当年官方文件为准；涉及软件操作时，以官方文档当前版本为准。

文件系统与命名

适用场景

你经常找不到文件；同一个项目有 `最终版`、`最终版2`、`真的最终版`；换电脑后路径全断；队友发来的压缩包解压出一堆乱码目录。

最小做法

先定一个能长期用的目录结构

建议按“年份 / 项目 / 类别”三层，不要按软件分：

```text
D:\work\
├─ 2026\
│  ├─ competition-a\
│  │  ├─ 01_docs\          # 申报书、说明、会议记录
│  │  ├─ 02_design\        # 图纸、模型、仿真
│  │  ├─ 03_code\          # 代码、脚本、配置
│  │  ├─ 04_data\          # 原始数据、处理结果
│  │  ├─ 05_output\        # 提交包、导出 PDF、答辩 PPT
│  │  └─ 99_archive\       # 旧版本、废弃方案
│  └─ course-project\
└─ shared\
```

规则：

1. 目录名用英文、数字、下划线，避免空格和中文标点。
2. 文件名用 `日期_主题_版本`，例如 `20261017_申报书_v3.docx`。
3. 原始数据只读，处理结果另存，不覆盖原文件。

4. 最终提交物放 `05_output`，不要和过程文件混在一起。

命名规则

- 日期统一 `YYYYMMDD`，排序即时间线。
- 版本用 `v1`、`v2`、`v3`，不要用 `最终`、`改好的`、`新新新`。
- 一个人负责的文件加缩写后缀，例如 `_lt`，但公开前去掉。
- 避免 `()`、`[]`、`#`、`&`、空格；这些字符在命令行和压缩包里容易出问题。

路径

- 代码里用相对路径，不要写 `C:\Users\<user>\...`。
- 路径含空格时，命令行要加引号：`cd "D:\my work\project"`。
- Windows 路径分隔符 `\` 在 Python 字符串里要转义，或者用 `r"D:\work"`、`Path("D:/work")`。
- 跨平台项目统一用 `/`，Python 的 `pathlib` 会自动处理。

快捷方式与固定入口

- 桌面只放当前项目的快捷方式，不要放文件本体。
- 常用目录固定到资源管理器“快速访问”。
- 给项目根目录放一个 `README.md`，写清目录用途和负责人。

进阶做法

- 用 `Everything` 建索引，按文件名秒搜；用 `ripgrep` 搜内容。
- 用 `tree /F` 或 `Get-ChildItem -Recurse` 导出目录树，放进项目文档。
- 给每个项目加 `.gitignore`，把缓存、数据、输出排除。
- 大文件单独放 `04_data`，不要塞进 Git。
- 定期跑一次“空文件夹、重复文件、超大文件”清理。

常见坑

- 把文件全放桌面，重装系统或换电脑时找不到。
- 用“最终版”命名，三周后不知道哪个是真的最终。
- 目录名带空格，脚本批量处理时路径被截断。
- 在 OneDrive 同步目录里跑 Git，两个系统同时改文件，冲突不断。
- 解压时不看目录结构，直接覆盖了原文件。
- 把原始数据和处理结果混在一起，出错后无法回退。

自检清单

- [ ] 我有一个固定的项目根目录。
- [ ] 目录按“年份/项目/类别”分层。
- [ ] 文件名带日期和版本，不用“最终版”。
- [ ] 原始数据只读，处理结果另存。
- [ ] 代码里没有本机绝对路径。
- [ ] 项目根目录有 README 说明目录用途。
- [ ] 桌面只放快捷方式，不放文件本体。

版本与时效

本文基于 Windows 11 与 PowerShell 7。`Everything`、`ripgrep` 的具体版本会变，但“固定目录 + 日期版本 + 相对路径”的原则不变。

备份与同步

适用场景

你只有一份文件放在桌面上；网盘同步出一堆“冲突副本”；移动硬盘坏了；误删后回收站也清空了。

最小做法

先分清“同步”和“备份”

- **同步**：让多台设备看到同一份最新文件。它会把删除和改错也同步过去。
- **备份**：保留历史版本，出错后能恢复。它不应该被一次误删同步掉。

OneDrive、百度网盘、坚果云默认是同步，不是备份。不要把“开了同步”当成“有备份”。

3-2-1 原则

- **3** 份数据：原件 + 两份副本。
- **2** 种介质：本机硬盘 + 移动硬盘/另一台电脑。
- **1** 份异地：网盘、机房、家里。

学生版最小可行：

1. 本机工作目录。
2. 移动硬盘每周备份一次。
3. 重要文件另传一份到网盘或学校云盘。

目录分层

```text
work\            # 正在改的文件
archive\         # 已完成的版本，按日期打包
backup\          # 移动硬盘/网盘副本
```

- `work` 可以频繁改。
- `archive` 只新增，不覆盖；每周把完成的项目打成 `20261017_project.zip`。
- `backup` 用同步工具自动传，不手动拖。

恢复演练

备份不做恢复演练，等于没有备份。每学期至少做一次：

1. 从移动硬盘/网盘下载一个旧项目。
2. 解压到新目录，确认文件完整、能打开。
3. 记录恢复耗时和缺失文件。

进阶做法

- 用 `robocopy` 做增量备份：

```powershell
robocopy "D:\work" "E:\backup\work" /MIR /R:2 /W:5 /LOG+:E:\backup\robocop
```

`/MIR` 会镜像删除，第一次用之前先确认目标目录没有别的东西。

- 用 Git 管理代码和文档，提交历史就是版本备份；大文件用 Git LFS 或单独存网盘。
- 用 7-Zip 打包时加密码和恢复记录，防止压缩包损坏。
- 给重要文件算哈希，恢复后校验：

```powershell
Get-FileHash .\submission.zip -Algorithm SHA256
```

常见坑

- 只开 OneDrive 同步，误删后同步删除，回收站也过期。
- 在同步目录里直接编辑 Office 文件，冲突副本越来越多。
- 把 Git 仓库放在 OneDrive 里，`.git` 被同步出冲突。
- 移动硬盘只插不备份，坏了才发现里面是唯一副本。
- 压缩包没有恢复记录，坏一个字节整个包打不开。
- 备份文件从不检查，真要用时发现是空的。

自检清单

- [ ] 我知道同步和备份的区别。
- [ ] 重要文件至少有 3 份、2 种介质、1 份异地。
- [ ] 每周有一次增量备份。
- [ ] 每月做一次恢复演练。
- [ ] Git 仓库不在 OneDrive 同步目录里。
- [ ] 压缩包有恢复记录，重要文件有哈希。
- [ ] 备份日志保留，知道上次成功时间。

版本与时效

本文基于 Windows 11、PowerShell 7、OneDrive。不同网盘的冲突处理策略不同，但“同步不等于备份”这一点不变。

编码、后缀与压缩包

适用场景

打开中文文本是乱码；CSV 用 Excel 打开变成一堆问号；代码在 Windows 正常、到 Linux 就报错；压缩包解压后文件名全是 `锟斤拷`。

最小做法

先分清三个东西

- **字符编码**：文字怎么变成字节。常用 UTF-8；国内旧文件常见 GBK/GB2312。
- **换行符**：Windows 是 `CRLF`（`\r\n`），Linux/macOS 是 `LF`（`\n`）。
- **文件后缀**：只是文件名的一部分，不代表内容。把 `.xlsx` 改成 `.zip` 不会自动转换格式。

统一用 UTF-8

- 新建文本、代码、Markdown、CSV 一律 UTF-8。
- 不要用“UTF-8 with BOM”，除非目标软件明确要求；BOM 会让某些脚本第一行解析失败。
- Excel 打开 CSV 乱码时，用“数据 → 从文本/CSV”导入，手动选 UTF-8，而不是双击。

换行统一 LF

- Git 仓库加 `.gitattributes`：

```text
* text=auto eol=lf
```

- Windows 编辑器（VS Code、Typora）把默认换行设为 LF。
- 老项目如果全是 CRLF，不要一次性全改，先确认构建/脚本是否依赖。

压缩包

- 通用格式用 `.zip`；需要高压缩率用 `.7z`。
- 打包前先整理目录，不要在压缩包里放一层“新建文件夹”。
- 大压缩包加恢复记录（7-Zip 的 `-rr5%`），坏一个字节还能修。
- 加密压缩包用强密码，密码单独通过另一渠道发送，不要和压缩包放一起。

```powershell
7z a -t7z -mx=9 -mhe=on -rr5% project.7z .\project
```

进阶做法

- 用 VS Code 右下角编码切换，另存为 UTF-8。
- 用 Python 检测编码：

```python
from pathlib import Path
raw = Path("data.csv").read_bytes()
for enc in ("utf-8-sig", "utf-8", "gbk", "gb18030"):
    try:
        print(enc, raw.decode(enc)[:80])
        break
    except UnicodeDecodeError:
        continue
```

- 用 `file` 命令（Linux/macOS）或 `Format-Hex`（PowerShell）看文件头。
- 压缩包解压先看目录树，再决定是否覆盖。
- 给压缩包算 SHA-256，传输后校验。

常见坑

- 把后缀当格式：`.doc` 改 `.docx` 打不开。
- Excel 双击 CSV，默认按本地编码解析，中文乱码。
- UTF-8 BOM 导致 Python/Shell 第一行报“invalid character”。
- CRLF/LF 混用，Git diff 整个文件都变红。
- 压缩包里的文件名用 GBK，解压到 UTF-8 系统变乱码。
- 加密压缩包和密码放在同一封邮件里。
- 压缩包没有恢复记录，网盘传输损坏后无法修复。

自检清单

- [ ] 新文件统一 UTF-8，无 BOM。
- [ ] Git 仓库有 `.gitattributes` 统一 LF。
- [ ] CSV 用导入方式打开，确认编码。
- [ ] 压缩包目录结构干净，没有多余层级。
- [ ] 大压缩包有恢复记录。
- [ ] 加密压缩包密码单独发送。
- [ ] 重要压缩包有 SHA-256 校验。

版本与时效

本文基于 Windows 11、VS Code、7-Zip 24.x。编码标准不变，软件菜单会变；遇到乱码先确认编码，不要反复改后缀。

终端与批量操作

适用场景

你要给 200 个文件改名、批量转图片格式、统计目录大小、查找重复文件，或者把手动点十遍的操作写成一条命令。

最小做法

先认识三个概念

- **当前目录**：命令默认在这里执行。`pwd` 查看，`cd` 切换。
- **管道**：把上一个命令的输出交给下一个命令。`|` 是管道符。
- **对象**：PowerShell 输出的是对象，不只是文本；可以用 `Select-Object`、`Where-Object`、`ForEach-Object` 处理。

常用命令

```powershell
Get-Location                    # 当前目录
Get-ChildItem                   # 列出文件
Get-ChildItem -Recurse          # 递归列出
Set-Location D:\work            # 切换目录
New-Item -ItemType Directory -Name demo    # 新建目录
Copy-Item a.txt b.txt           # 复制
Move-Item a.txt archive\        # 移动
Remove-Item a.txt               # 删除（不进回收站，慎用）
Get-Content a.txt               # 读文本
Select-String "关键词" *.md   # 搜内容
```

批量重命名

先预览，再执行：

```powershell
Get-ChildItem *.jpg | ForEach-Object {
    $new = "2026_" + $_.Name
    Write-Host "$($_.Name) → $new"
}
```

确认无误后加 `Rename-Item`：

```powershell
Get-ChildItem *.jpg | ForEach-Object {
    Rename-Item $_.FullName ("2026_" + $_.Name)
}
```

批量统计

```powershell
Get-ChildItem -Recurse -File |
  Measure-Object -Property Length -Sum |
  Select-Object Count, @{n='GB';e={[math]::Round($_.Sum/1GB,2)}}
```

查找大文件

```powershell
Get-ChildItem -Recurse -File |
  Sort-Object Length -Descending |
  Select-Object -First 20 FullName, @{n='MB';e={[math]::Round($_.Length/1M
```

进阶做法

- 把常用命令写成 `.ps1` 脚本，放 `tools\`，加注释和参数。
- 用 `-WhatIf` 预览危险操作（支持的命令）：

```powershell
Remove-Item *.tmp -WhatIf
```

- 用 `Tee-Object` 同时输出到屏幕和日志。
- 用 `Start-Transcript` 记录整个终端会话。
- 用 `robocopy` 做批量复制/镜像，比 `Copy-Item` 更适合大目录。

常见坑

- `Remove-Item` 默认不进回收站，删错很难恢复。先 `-WhatIf`，再执行。
- 路径含空格不加引号，命令被拆成两段。
- 在 `C:\Windows\System32` 下执行脚本，权限不足或改错系统文件。
- 用 `*` 通配符前不预览，批量改错全部文件。
- 直接运行来路不明的 `.ps1`，可能被执行策略或恶意脚本拦住/利用。
- 在 OneDrive 同步目录里批量改名，同步冲突翻倍。

自检清单

- [ ] 我知道当前目录是什么。
- [ ] 批量操作前先用 `Write-Host` 或 `-WhatIf` 预览。
- [ ] 删除操作前确认有备份。
- [ ] 脚本放在项目 `tools\` 下，不进系统目录。
- [ ] 路径含空格时加了引号。
- [ ] 重要操作有日志。
- [ ] 陌生脚本先读内容，再决定是否执行。

版本与时效

本文基于 PowerShell 7 与 Windows 11。PowerShell 5.1 的语法基本兼容，但部分参数不同；`robocopy` 行为长期稳定。

浏览器插件与脚本风险

适用场景

你装了很多浏览器插件，网页变慢、弹广告；有人发来“油猴脚本”说能自动刷课/解锁功能；登录学校系统时担心插件读取密码。

最小做法

插件最小化

只保留三类：

1. 必需：密码管理器、翻译、文献抓取。
2. 高频：广告拦截、阅读模式。
3. 开发：调试工具，用完关掉。

其余插件停用或卸载。插件不是越多越好，每个插件都能读你打开的网页。

看权限

安装前看它要求什么权限：

- “读取和更改所有网站上的数据”：权限很大，谨慎。
- “读取浏览历史”：不需要就别给。
- “管理下载”：下载类插件才需要。
- “读取剪贴板”：很少有必要。

权限和功能不匹配的插件，不装。

油猴脚本的风险

油猴（Tampermonkey）本身是脚本管理器，风险来自脚本：

- 脚本可以读页面内容、发请求、改按钮。
- 来路不明的脚本可能偷 cookie、token、表单内容。
- “自动刷课/自动答题/解锁会员”类脚本，除了安全风险，还有学术诚信和封号风险。

原则：只装能看懂、来源可查、更新活跃的脚本；涉及账号、支付、学校系统的页面，关掉脚本再操作。

账号隔离

- 学校/银行/邮箱等重要账号，用一个干净的浏览器配置文件，不装杂七杂八插件。
- 日常浏览用另一个配置文件。
- 密码管理器开自动填充，但重要操作手动确认域名。

进阶做法

- Chrome/Edge 建多个配置文件：`School`、`Daily`、`Dev`。
- 用 `chrome://extensions` 查看每个插件的“详细信息 → 网站访问权限”，改成“点击时”。
- 定期导出插件清单，重装系统后按需恢复。
- 用浏览器自带的增强保护/安全浏览，但不要替代密码管理器。
- 涉及登录的脚本，先在无痕窗口测试，确认不会自动提交表单。

常见坑

- 装“免费 VPN/加速器”插件，实际在卖流量或注入广告。
- 给插件“所有网站”权限，登录页也被读取。
- 用来路不明的油猴脚本刷课，账号被封或密码泄露。
- 插件长期不更新，漏洞没人修。

- 在装了十几个插件的浏览器里登录学校系统，出问题难排查。
- 把密码保存在浏览器明文/弱同步里，不开主密码。

自检清单

- [ ] 我只保留必需、高频、开发三类插件。
- [ ] 每个插件的权限和功能匹配。
- [ ] 重要账号用独立浏览器配置文件。
- [ ] 油猴脚本来源可查，能看懂关键逻辑。
- [ ] 登录/支付页面关闭脚本和可疑插件。
- [ ] 密码管理器开启，重要操作手动确认域名。
- [ ] 定期清理不用的插件。

版本与时效

本文基于 Chrome/Edge 2026 版。插件名称和菜单会变，但“权限最小化 + 账号隔离 + 脚本审慎”的原则不变。

磁盘空间与软件清理

适用场景

C 盘爆红；软件装完找不到卸载入口；想扩容分区但怕丢数据；清理完发现误删了项目文件。

最小做法

先看空间去哪了

```powershell
Get-PSDrive -PSProvider FileSystem | Select-Object Name, Used, Free
```

找大目录：

```powershell
Get-ChildItem C:\ -Directory -Force -ErrorAction SilentlyContinue |
  ForEach-Object {
    $size = (Get-ChildItem $_.FullName -Recurse -File -ErrorAction Silentl
             Measure-Object Length -Sum).Sum
    [PSCustomObject]@{ Path=$_.FullName; GB=[math]::Round($size/1GB,2) }
  } | Sort-Object GB -Descending | Select-Object -First 15
```

安全清理顺序

1. 回收站。
2. 浏览器缓存、下载目录。
3. 软件缓存（微信/QQ/网盘/IDE）。
4. Windows 更新缓存（用系统“磁盘清理”）。

5. 不用的软件。
6. 最后才考虑分区调整。

不要直接删 `C:\Windows`、`Program Files`、`Users\<user>\AppData` 里不认识的目录。

卸载软件

- 优先用软件自带卸载器。
- 卸载后用 Revo Uninstaller 等工具扫残留（可选）。
- 卸载前确认没有项目依赖它；比如 Python 环境、CUDA、驱动。
- 破解软件、来路不明的“绿色版”不要装，卸载也可能卸不干净。

分区与扩容

- 分区调整前必须备份。
- Windows 磁盘管理只能把相邻的未分配空间合并到分区。
- 跨分区调整用 DiskGenius/分区助手，操作前断电风险要考虑。
- C 盘扩容优先“压缩相邻 D 盘 → 扩展 C 盘”，不要跨物理磁盘。
- BitLocker/设备加密开启时，先暂停保护再操作。

进阶做法

- 用 `WizTree` / `TreeSize` 可视化空间占用，比递归命令快。
- 把大文件（数据集、视频、模型权重）移到数据盘，用符号链接接回：

```powershell
New-Item -ItemType SymbolicLink -Path "D:\work\data" -Target "E:\datasets\
```

- 给软件安装目录统一到 `D:\apps`，避免 C 盘越来越满。
- 用 `winget` 管理软件安装/升级：

```powershell
winget list
winget upgrade --all
```

- 定期导出已安装软件清单，重装系统后按需恢复。

常见坑

- 直接删 `AppData`，软件配置全丢。
- 用“一键清理”工具删注册表/系统文件，系统不稳定。
- 分区调整中途断电，分区表损坏。
- 把项目放在 C 盘桌面，重装系统一起没。
- 卸载 Python/驱动后，依赖它的项目全部跑不起来。
- 只看“可用空间”，不看“临时文件/休眠文件/虚拟内存”。

自检清单

- [ ] 我知道空间被哪些目录占用。
- [ ] 清理前有备份，重要文件不在 C 盘桌面。
- [ ] 卸载软件前确认没有项目依赖。
- [ ] 分区操作前备份并接通电源。
- [ ] 不使用来路不明的“一键清理/破解”工具。
- [ ] 大文件放数据盘，用符号链接接回。
- [ ] 定期更新软件，保留安装清单。

版本与时效

本文基于 Windows 11、PowerShell 7、winget。分区工具和系统菜单会变，但“先备份、再动分区”的顺序不变。

信息甄别

适用场景

你搜到一个“学长说”、一张截图、一段 AI 总结，看起来很有道理，但不确定能不能信；政策一年一变，旧教程还在流传。

最小做法

先问四个问题

1. 谁说的？有名字、机构、出处吗？
2. 什么时候说的？政策/软件版本是哪一年的？
3. 原始出处在哪？能不能点进原文？
4. 有没有利益关系？卖课、卖资料、引流、带节奏？

四个问题里有两个答不上来，先别用。

三类高风险信息

AI 生成内容

- 可能编造论文、作者、DOI、政策条款。
- 可能把旧信息说得像新的。
- 可能漏掉限定条件，把“部分情况”说成“所有情况”。

用法：AI 适合扩展关键词、解释概念、整理笔记；事实、数字、政策必须回到原文。

营销号/培训广告

- 标题夸张：“只要三步”“保研必看”“内部渠道”。

- 内容模板化，最后导向课程/资料/社群。
- 截图可能来自别处，数据可能伪造。

用法：当线索，不当依据。

过期政策/教程

- 学校政策按学年变，竞赛规则按届变，软件界面按版本变。
- 旧教程的菜单、链接、截止时间可能全错。

用法：找到“适用年份/版本”，找不到就默认可能过期。

截图与二手信息

- 截图可以裁剪、拼接、改字。
- 聊天记录可以伪造。
- “别人说”不能作为正式依据。

需要用时：找原始文件/官网/邮件，至少两个独立来源交叉验证。

进阶做法

- 用“来源分级”：官方文件 > 一手论文/文档 > 机构教程 > 个人博客 > 营销号。
- 用“反向搜索”查图片/截图最早出处。
- 用“站内搜索”查官网/公众号历史通知。
- 用“日期过滤”排除旧版本。
- 保存链接、标题、作者、日期，写材料时能回溯。

常见坑

- 把 AI 总结当原文，引用不存在的文献。
- 看到高赞回答就直接用。
- 用去年的竞赛认定文件判断今年。

- 截图里的“通知”没有文号/盖章。
- 只看结论，不看样本量和适用条件。
- 把培训机构的宣传当成学校政策。

自检清单

- [ ] 我知道信息是谁说的、什么时候说的。
- [ ] 我找到了原始出处。
- [ ] 关键结论有两个独立来源。
- [ ] 我确认了适用年份/软件版本。
- [ ] 我没有把 AI 摘要当原文。
- [ ] 我没有把营销号当政策。
- [ ] 我保存了链接和日期。

版本与时效

本文的方法不依赖具体平台。政策以当年官方文件为准，软件以官方文档当前版本为准；任何二手信息都可能过期。

卷二 · 文档力

竞赛、课程、科研，最后都要落到一份可交付的文件上。文档力不是“会用 Word”，而是：

- 知道一份文档应该有哪些结构。
- 知道样式、编号、题注、交叉引用怎么设，改内容不用手工重排。
- 知道什么内容该放正文，什么该放附录。
- 知道导出 PDF 前要检查什么。

本卷章节

- Word 排版最小工作流：样式、多级列表、题注、交叉引用、目录、导出。
- Excel 数据表与图表：表结构、函数、透视表、图表选择。
- PPT 答辩叙事：结构、每页一个结论、证据链、问答。
- Markdown 与编辑器：语法、VS Code/Typora、图片路径、导出。
- Mermaid 与 draw.io 图示：流程图、时序图、架构图、导出。
- 排版、打印与 PDF：页边距、字体、页码、导出与打印检查。
- 文档协作与版本：Word 修订、在线文档、Git、冲突处理。

先读哪篇

第一次写申报书或论文，先读Word 排版最小工作流。它不追求排版漂亮，先保证结构正确、能改、能导出。

Word 排版最小工作流

适用场景

你要写课程报告、竞赛申报书、论文或项目结题材料。文档会反复修改，图、表、章节会增删，最后还要导出 PDF。

最小做法

第一步：先定样式，不先写正文

打开 Word，先把这几类样式定好：

样式	用途	关键设置
标题 1	一级标题	黑体/加粗，段前段后，与下段同页
标题 2	二级标题	字号递减，编号跟随
标题 3	三级标题	尽量少用
正文	主体文字	首行缩进 2 字符，1.5 倍行距
图注	图下方说明	小一号字，居中
表注	表上方说明	小一号字，居中
公式	独立公式	居中，编号右对齐

样式定好后，写作时只点样式，不用手动调字号、加粗、空格。

第二步：用多级列表管章节编号

1. 在“开始”里点“多级列表” → “定义新的多级列表”。
2. 把级别 1 链接到“标题 1”，级别 2 链接到“标题 2”。
3. 编号格式设为 `1`、`1.1`、`1.1.1`。
4. 正文里的“第一章” “1.1”不要手打，交给列表。

这样插入或删除章节时，编号会自动更新。

第三步：图、表、公式用题注

- 选中图 → 引用 → 插入题注 → 新建标签“图”。
- 表：题注放在表上方；图：题注放在图下方。
- 公式：用“插入 → 公式”，编号可以用表格或制表位实现。

题注的作用不是好看，是让交叉引用能自动更新。

第四步：正文引用用交叉引用

在正文里写“如图 3 所示”时：

1. 引用 → 交叉引用 → 引用类型选“图”。
2. 选择对应题注，插入。
3. 图号变化后，全选 → F9 更新域。

不要手打“图 3”。手打的编号，插入一张图就会全乱。

第五步：三线表

学术文档常用三线表：

1. 插入表格，填内容。
2. 选中表格 → 边框 → 无框线。
3. 只保留顶线、表头下线、底线。
4. 表注放在表格上方，写清单位、样本量、显著性标记。

如果学校/期刊有模板，优先用模板，不要自创格式。

第六步：目录与 PDF

1. 正文标题必须用“标题 1/2/3”样式。
2. 引用 → 目录 → 自动目录。
3. 导出前：Ctrl+A → F9 → 更新整个目录。
4. 文件 → 导出 → PDF，勾选“创建书签时使用标题”。
5. 打开 PDF 检查：目录页码、图表编号、公式编号、分页、字体是否嵌入。

进阶做法

- 用“样式集”统一全文档，不要逐个改。
- 给长文档加“分节符”，不同章节用不同页眉页脚。
- 用“修订”模式改稿，接受后再导出。
- 图片统一 300 DPI，宽度不超过版心。
- 公式用 Word 自带公式编辑器，方便后期修改。

常见坑

- 用空格/回车对齐，改字号就全乱。
- 手动打章节号、图号、表号，插入内容后编号错位。
- 从网页复制内容，带入一堆隐藏格式。
- 目录不更新就导出，页码对不上。
- 修订没接受、批注没删除，直接交稿。
- 图片是截图，放大后模糊。
- 字体未嵌入，别人打开 PDF 变宋体/乱码。

自检清单

- [ ] 全文只用样式控制标题和正文。

- [ ] 章节编号由多级列表生成。
- [ ] 图、表、公式都有题注。
- [ ] 正文引用使用交叉引用，没有手打编号。
- [ ] 目录已更新，页码正确。
- [ ] 修订已接受，批注已删除。
- [ ] PDF 已检查分页、图表、字体。
- [ ] 文件属性里没有不该出现的作者/单位信息（公开前必查）。

版本与时效

本文基于 Microsoft Word 2024（16.0）界面。Word 2016/2019/365 的菜单名称基本一致，位置可能略有差异。WPS 的操作路径不同，但“样式—题注—交叉引用—目录”的逻辑相同。

Excel 数据表与图表

适用场景

你要整理实验数据、做分工表、统计报名信息、画对比图，或者把一堆 CSV 合并成一张表。

最小做法

数据表先定结构

一张表只做一件事，第一行是表头，每列一个字段：

日期	项目	负责人	数值	单位	备注
2026-10-01	A	张三	12.3	mm	
2026-10-02	A	李四	12.5	mm	

规则：

- 不合并单元格。
- 不在表头上面加标题行。
- 一列只放一种数据类型。
- 空值留空，不要写 `无`、`N/A`、`-` 混用。
- 原始数据单独一个 sheet，处理过程另开 sheet。

常用函数

```excel
=SUM(B2:B100)
=AVERAGE(B2:B100)
=MAX(B2:B100)
=COUNTIF(C2:C100,"张三")
=SUMIF(C2:C100,"张三",D2:D100)
=IFERROR(VLOOKUP(A2,Sheet2!A:B,2,FALSE),"未找到")
=TEXT(A2,"yyyy-mm-dd")
=ROUND(D2,2)
```

- `VLOOKUP` 只适合从左往右查；复杂查询用 `XLOOKUP`（新版 Excel）。
- 查找值两边格式要一致：文本 vs 数字、前后空格都会导致 `#N/A`。
- 用 `TRIM` 去空格，用 `VALUE` 转数字。

透视表

1. 选中数据区域 → 插入 → 数据透视表。
2. 行：分类字段；列：对比字段；值：求和/计数/平均。
3. 原始数据更新后，右键透视表 → 刷新。

透视表适合“按类别汇总”，不适合直接当最终排版表。

图表选择

目的	图表
比较类别	柱状图
看趋势	折线图
看分布	直方图/箱线图
看占比	饼图（类别少时）
看两个变量关系	散点图

规则：

- 一张图只表达一个结论。
- 坐标轴有标题和单位。
- 图例、数据标签不要遮挡。
- 颜色不超过 5 种，重点用对比色。
- 导出前把图表区域调成合适大小，不要直接截图。

进阶做法

- 用“表格”功能（Ctrl+T）让公式自动扩展。
- 用“数据验证”限制输入，减少脏数据。
- 用“条件格式”标出异常值。
- 用 Power Query 合并多个 CSV/Excel，步骤可复用。
- 用 `pandas` 处理超过几万行的数据，Excel 只做展示。

常见坑

- 合并单元格导致排序、筛选、透视表全乱。
- 数字被存成文本，求和为 0。
- 日期格式不统一，按时间排序失效。
- 图表用 3D、渐变、阴影，读者看不清数据。
- 直接改原始数据，出错无法回溯。
- 文件里藏着批注、修订、个人信息，公开前没清理。

自检清单

- [ ] 一张表只做一件事，第一行是表头。
- [ ] 没有合并单元格。
- [ ] 一列只有一种数据类型。
- [ ] 原始数据和处理结果分开 sheet。

- [ ] 图表有标题、坐标轴、单位。
- [ ] 图表颜色不超过 5 种，重点突出。
- [ ] 公开前清理批注、修订、个人信息。

版本与时效

本文基于 Microsoft Excel 2024。`XLOOKUP`、Power Query 在旧版可能不可用；WPS 的函数名称基本兼容，但菜单位置不同。

PPT 答辩叙事

适用场景

你要做课程汇报、竞赛答辩、项目结题。内容很多，时间很短，评委只看几分钟。

最小做法

先定叙事，不先找模板

答辩不是“把做过的事全放上去”，而是回答四个问题：

1. 你要解决什么问题？
2. 为什么你的方案值得做？
3. 你怎么证明它有效？
4. 下一步是什么？

推荐结构（10-15 分钟）：

1. 封面：项目名 + 一句话定位。
2. 问题：场景、痛点、现有方案缺什么。
3. 目标：可验证的指标。
4. 方案：总体路线图 + 关键模块。
5. 证据：实验/原型/对比结果。
6. 分工与进度：谁做了什么，下一步。
7. 风险与应对：至少三条。
8. 结束页：结论 + 联系方式。

每页只讲一个结论

- 标题写结论，不写“方案介绍”“实验结果”这种空标题。
- 一页最多一个图 + 三条要点。
- 图放大，字少；评委看不清的图等于没放。
- 关键数字加粗，单位写清。

证据链

每个结论后面跟一个证据：

- 实验数据：样本量、条件、误差。
- 对比：和谁比、指标是什么、提升多少。
- 原型：照片/视频/现场演示。
- 失败案例：边界条件、未达标项，主动说。

没有证据的“创新点”，改成“设计思路”或删掉。

讲稿

- 每页写 2-4 句口语，不写逐字稿。
- 开头 30 秒说清“我们做了什么、结果如何”。
- 时间分配：问题 20%，方案 30%，证据 35%，收尾 15%。
- 至少完整试讲两遍，掐表。

问答

提前准备 5 个问题：

- 你的方案和现有方案比，优势在哪？
- 关键指标怎么测的？误差多大？
- 如果某个模块失败，怎么办？
- 成本/周期/可制造性如何？
- 哪些是你们做的，哪些是现成工具？

回答原则：先结论，再证据，最后边界。不会就说“这一点还没验证”。

进阶做法

- 用母版统一字体、颜色、页码。
- 图表用矢量图或高分辨率 PNG，不要截图。
- 关键页加“备份页”，放在最后，问答时跳转。
- 现场演示准备录屏 + 本地文件 + 备用设备。
- 用演讲者视图看备注，但不要照读。

常见坑

- 第一页放 20 个项目背景，评委不知道你要讲什么。
- 每页字太多，字号小于 18pt。
- 动画过多，翻页卡顿。
- 只有方案，没有证据。
- 现场演示依赖网络，断网就讲不下去。
- 回答问题超时，挤掉后面的内容。
- 结束页写“谢谢聆听”，没有结论。

自检清单

- [ ] 每页标题是一个结论，不是栏目名。
- [ ] 一页最多一个图 + 三条要点。
- [ ] 每个结论有证据或明确标注“待验证”。
- [ ] 讲稿能在规定时间内讲完。
- [ ] 试讲至少两遍，掐表。
- [ ] 准备了 5 个问答。
- [ ] 演示有录屏和本地备份。
- [ ] 结束页有结论和联系方式。

版本与时效

本文基于 PowerPoint 2024。模板和动画会变，但“问题—方案—证据—边界”的叙事结构不变。

Markdown 与编辑器

适用场景

你要写实验记录、项目文档、博客、README，或者把 Word 里的内容整理成可版本管理的文本。

最小做法

常用语法

markdown

```markdown
# 一级标题
## 二级标题

**加粗**  *斜体*  `行内代码`

- 无序列表
1. 有序列表

> 引用

[链接](https://example.com)
![图片](assets/demo.png)

| 列 1 | 列 2 |
|---|---|
| A | B |

```python
print("hello")
```
```

````
规则：

- 标题层级不要跳级：`#` → `##` → `###`。
- 代码块标语言，方便高亮。
- 图片放 `assets/` 或同目录，用相对路径。
- 文件名用英文/数字/短横线，避免空格。
- 一段只讲一件事，长段落拆成列表。

### 编辑器选择

- **VS Code**：适合代码 + Markdown + Git，插件多，免费。
- **Typora**：所见即所得，适合纯写作；注意许可证。
- **Obsidian/Logseq**：适合双链笔记，不适合当代码仓库。
- **在线编辑器**：方便协作，但隐私和导出格式要确认。

### VS Code 最小配置

```json
{
  "files.encoding": "utf8",
  "files.eol": "\n",
  "editor.wordWrap": "on",
  "markdown.preview.breaks": true,
  "markdownlint.enabled": true
}
```
````

推荐插件：Markdown All in One、markdownlint、Code Spell Checker、GitLens。

导出

- 导出 PDF：用 VS Code 的 Markdown PDF 插件，或先转 HTML 再打印。
- 导出 Word：用 Pandoc；没有 LaTeX 时用 `--reference-doc` 套模板。
- 导出前检查：图片路径、代码块换行、表格宽度、中文字体。

进阶做法

- 用 frontmatter 记录标题、日期、标签：

```yaml
---
title: 实验记录
date: 2026-10-17
tags: [experiment, motor]
---
```

- 用 `<!-- comment -->` 写不渲染的备注。
- 用 Mermaid 画流程图，文本可 diff。
- 用 Git 管理文档，提交信息写“改了什么”。
- 用 `markdownlint` 统一格式，减少无意义 diff。

常见坑

- 图片用绝对路径，换电脑全裂。
- 标题层级乱跳，目录生成错。
- 代码块不标语言，高亮和复制都难受。
- 表格用空格对齐，渲染后错位。
- 直接粘贴网页内容，带入隐藏格式。
- 中文文件名在部分静态站点/命令行下编码出错。
- 导出 PDF 时字体缺失，中文变方块。

自检清单

- [ ] 标题层级不跳级。
- [ ] 代码块标了语言。
- [ ] 图片用相对路径，文件存在。

- [ ] 文件名不含空格和特殊字符。
- [ ] 表格用标准 Markdown 语法。
- [ ] 导出 PDF/Word 后检查中文、图片、分页。
- [ ] 文档纳入 Git，提交信息可读。

版本与时效

本文基于 VS Code 1.141、Typora 1.8。Markdown 语法稳定，编辑器和插件会变；导出前以实际渲染结果为准。

Mermaid 与 draw.io 图示

适用场景

你要在文档/答辩里画流程、系统架构、任务时序、状态机；用截图或手绘图，改一次就要重画。

最小做法

Mermaid：文本画图

流程图：

```mermaid
flowchart LR
    A[需求] ⟶ B[方案]
    B ⟶ C{可行?}
    C -- 是 ⟶ D[原型]
    C -- 否 ⟶ B
    D ⟶ E[测试]
    E ⟶ F[交付]
```

时序图：

```mermaid
sequenceDiagram
    participant U as 用户
    participant S as 系统
    participant D as 数据库
    U->>S: 提交请求
    S->>D: 查询
    D-->>S: 返回结果
    S-->>U: 响应
```

状态图：

```mermaid
stateDiagram-v2
    [*] --> 待开始
    待开始 --> 进行中
    进行中 --> 阻塞
    阻塞 --> 进行中
    进行中 --> 已完成
    已完成 --> [*]
```

规则：

- 一张图只表达一个结构。
- 节点文字短，动词开头。
- 方向统一：从左到右或从上到下。
- 颜色不超过 3 种，重点节点加粗。

draw.io：拖拽画图

适合：系统架构、硬件框图、流程排版要求高的图。

步骤：

1. 新建空白图，选横向/纵向。
2. 先放主流程，再补分支。
3. 用对齐/分布工具，不要手动对。
4. 统一字体、字号、线宽。

5. 导出 SVG（可编辑）或 PNG（300 DPI）。

进阶做法

- Mermaid 放 Markdown 里，Git 可 diff，改文字就改图。
- 复杂架构用 draw.io，导出 SVG 嵌进文档。
- 给图加图注：图号、标题、数据来源。
- 用子图（subgraph）分组，减少连线交叉。
- 答辩图放大到一页一个，不要塞 10 个节点。

常见坑

- 图里文字太小，投影看不清。
- 连线交叉，读者跟着线跑丢。
- 一张图同时画流程、架构、数据流，信息过载。
- 用截图代替源文件，改一个字要重画。
- 颜色太多，重点不突出。
- 导出 PNG 分辨率不够，打印模糊。
- Mermaid 语法在 GitHub/本地编辑器渲染不一致。

自检清单

- [ ] 一张图只表达一个结构。
- [ ] 节点文字短，方向统一。
- [ ] 颜色不超过 3 种，重点突出。
- [ ] 图有图号和标题。
- [ ] 源文件保留（`.mmd`、`.drawio`）。
- [ ] 导出 SVG 或 300 DPI PNG。
- [ ] 投影/打印后能看清。

版本与时效

本文基于 Mermaid 11、draw.io 24。语法和导出选项会变，但“源文件 + 矢量导出 + 一张图一个结论”的原则不变。

排版、打印与 PDF

适用场景

文档在电脑上好看，打印/导出后分页乱、字体变、图表被截断；提交系统只收 PDF；论文格式审查总被打回。

最小做法

先看要求

- 学校/竞赛/期刊有没有模板？有就先用模板，不要自创。
- 页边距、字体、字号、行距、页码位置，逐项对照。
- 摘要/正文/参考文献的格式可能不同。
- 文件名、文件大小、页数限制。

页面设置

- A4，页边距按模板：常见上下 2.54 cm、左右 3.17 cm，或学校指定值。
- 正文宋体/仿宋，标题黑体，英文 Times New Roman；以模板为准。
- 行距 1.5 倍或固定值 20-22 pt。
- 页码从正文开始，封面/目录不编号。
- 页眉写论文/项目名，页脚写页码。

分页与分节

- 封面、目录、正文、附录之间用“分节符（下一页）”，不要连按回车。
- 不同节可以设不同页眉/页码。
- 图表不要跨页；表格跨页时重复表头。

- 标题不要单独留在页底（与下段同页）。

导出 PDF

1. 更新目录、交叉引用、图表编号。
2. 接受所有修订，删除批注。
3. 文件 → 导出 → PDF，勾选“创建书签时使用标题”。
4. 检查：字体嵌入、分页、图表、公式、页码、超链接。
5. 用另一台设备/阅读器打开一次。

打印检查

- 先打一份黑白草稿，检查灰度对比。
- 彩色图打印后是否还能区分。
- 装订线是否留够。
- 双面打印时奇偶页页边距是否对称。

进阶做法

- 用样式统一全文，不要手动调。
- 用“域”做页码、目录、交叉引用，改内容自动更新。
- 长文档拆成多个子文档，最后合并。
- 用 PDF 工具压缩图片，控制文件大小。
- 提交前用 `pdfinfo` /阅读器检查页数、字体、元数据。

常见坑

- 用回车/空格排版，改一个字全乱。
- 目录没更新，页码对不上。
- 字体没嵌入，别人打开变宋体/乱码。
- 图表跨页，读者要翻页对照。

- 修订/批注没删，直接交稿。
- 文件名含空格/中文，系统上传失败。
- PDF 是扫描件，文字不可检索。

自检清单

- [ ] 对照了学校/竞赛/期刊模板。
- [ ] 页边距、字体、行距、页码符合要求。
- [ ] 封面/目录/正文分节，页码正确。
- [ ] 图表不跨页，表头可重复。
- [ ] 目录、交叉引用已更新。
- [ ] 修订已接受，批注已删除。
- [ ] PDF 字体嵌入、分页、书签正常。
- [ ] 换设备打开检查过。

版本与时效

本文基于 Word 2024 与常见 PDF 阅读器。模板要求每年可能变，以当年官方文件为准。

文档协作与版本

适用场景

三个人改同一份申报书；队友发来“最终版”和“最终版_改”；改完不知道谁动了哪一段；提交前合并出一堆冲突。

最小做法

先定“唯一主文件”

- 指定一个人维护主文件，其他人提交修改建议。
- 主文件放共享目录/仓库，文件名固定，不加“最终版”。
- 每次修改前先拉最新版，改完立刻回传。

Word 修订

1. 审阅 → 修订，开启 tracked changes。
2. 改完保存，文件名带日期和修改人：`20261017_申报书_张三修订.docx`。
3. 主文件维护者逐条接受/拒绝，不要整篇接受。
4. 合并后删除批注、隐藏个人信息。

在线文档

- 腾讯文档/飞书/石墨适合实时协作，但导出格式可能变。
- 定好“谁负责哪一节”，避免多人同时改同一段。
- 用评论而不是直接改别人段落。
- 重要版本导出 PDF 存档。

Git 管文档

适合 Markdown/代码/配置，不适合二进制 Word 的频繁合并。

```powershell
git pull
git add docs/
git commit -m "docs: update budget section"
git push
```

规则：

- 一次提交只改一个主题。
- 提交信息写“改了什么”。
- 二进制文件（docx/pptx）不要多人同时改；用“谁改谁提交”。
- 大文件用网盘，不进 Git。

进阶做法

- 用 `docs/` 分目录：`01_申报书/`、`02_附件/`、`03_答辩/`。
- 用 `CHANGELOG.md` 记录每次提交给谁、改了什么。
- 用分支：`draft/申报书`、`final/提交版`。
- 用 Issue 跟踪待改项，改完关闭。
- 用 PDF 作为对外唯一版本，Word 只作编辑源。

常见坑

- 多人同时改同一个 docx，最后只能手动合并。
- 文件名“最终版”满天飞，不知道哪个是提交版。
- 在线文档导出后格式错乱。
- Git 里提交大二进制文件，仓库爆炸。
- 修订没接受，打印出来带删除线。

- 批注里写了队友/老师信息，公开前没清理。
- 没有版本记录，出问题无法回退。

自检清单

- [ ] 有唯一主文件和维护人。
- [ ] 修改前先拉最新版。
- [ ] Word 修订逐条处理，批注删除。
- [ ] 文件名带日期/修改人，不用“最终版”。
- [ ] 对外版本统一导出 PDF。
- [ ] Git 提交信息可读，二进制文件不频繁合并。
- [ ] 有 CHANGELOG 或提交记录。

版本与时效

本文基于 Word 2024、腾讯文档/飞书、Git 2.55。协作工具会变，但“唯一主文件 + 明确分工 + 版本记录”的原则不变。

卷三・代码与数据力

不是只有计算机专业才需要代码。机械、自动化、材料、能源，只要涉及数据、仿真、控制、图像，就绕不开：

- 环境能不能重装。
- 结果能不能复现。
- 代码能不能被别人接手。
- 版本能不能回退。

本卷章节

- Git 第一次提交：从 `git init` 到远程私有仓库。
- Python 环境与依赖：虚拟环境、conda/uv、依赖锁定。
- GitHub 私有仓库与协作：分支、Issue、PR、Release、Actions。
- 数据清洗与可复现：原始数据只读、清洗脚本、随机种子、复现清单。
- 可视化与出图：图表选择、配色、坐标轴、图注、导出。
- VS Code 配置：工作区、解释器、插件最小集、远程开发。
- 代码质量与测试：结构、命名、注释、日志、单元测试。
- AI 辅助编程边界：能做什么、不能做什么、怎么审查、怎么声明。

先读哪篇

如果你还没用过 Git，先读Git 第一次提交。如果你已经在写代码但结果老是复现不了，先补“Python 环境与依赖”和“数据”。

Git 第一次提交

适用场景

你开始写代码、写文档，或者要跟队友协作。你担心改坏了回不去，或者想把版本传到 GitHub 私有仓库。

最小做法

第一步：配置身份

```bash
git config --global user.name "你的名字或 ID"
git config --global user.email "你的邮箱"
git config --global init.defaultBranch main
```

如果要把仓库公开，建议用 GitHub 提供的 noreply 邮箱，避免私人邮箱进入提交历史。

第二步：初始化仓库

```bash
mkdir my-project
cd my-project
git init
```

第三步：看状态、加文件、提交

```bash
git status
git add README.md
git commit -m "docs: add README"
git log --oneline
```

一次提交只做一件事。提交信息写“做了什么”，不要写“update”。

第四步：写 `.gitignore`

至少忽略：

```text
node_modules/
__pycache__/
*.pyc
.env
.venv/
venv/
*.log
.DS_Store
Thumbs.db
```

密钥、密码、数据库文件、大模型权重、几十 MB 的安装包，都不要进 Git。

第五步：连到远程私有仓库

在 GitHub 新建私有仓库，不要勾选自动生成 README（避免和本地冲突），然后：

```bash
git remote add origin https://github.com/<user>/<repo>.git
git branch -M main
git push -u origin main
```

之后每次：

```bash
git add <文件>
git commit -m "feat: ..."
git push
```

第六步：回退

- 还没提交的修改：`git restore <文件>`
- 已提交但想撤销提交、保留修改：`git reset --soft HEAD~1`
- 想看某次提交改了什么：`git show <commit>`

不要用 `git reset --hard` 除非你确定不要那些修改。

进阶做法

- 用分支做实验：`git switch -c experiment`，失败就删分支。
- 用 `.gitattributes` 统一换行，避免 Windows/macOS 协作时整文件 diff。
- 大文件用 Git LFS，不要直接提交。
- 提交前跑测试，CI 通过再合并。
- 用 `git log --graph --oneline --all` 看分支结构。

常见坑

- 把 `.env`、密钥、token 提交进去。即使后来删除，历史里仍然存在。
- 在 OneDrive 同步目录里建 Git 仓库，两个系统同时改文件，容易冲突。
- 直接删 `.git` 目录“重置”，历史全部丢失。
- 把 `node_modules/`、`venv/`、数据集提交进去，仓库变成几个 GB。
- 提交信息只写“update” “修改”，三个月后自己都看不懂。
- 多人同时改同一个文件，不会处理冲突，直接覆盖队友的版本。

自检清单

- [ ] `git config user.name` 和 `user.email` 正确。
- [ ] `.gitignore` 覆盖了依赖、环境、密钥、日志。
- [ ] 第一次提交只包含必要文件。
- [ ] 远程仓库是私有的（公开前）。
- [ ] 提交信息能看懂“做了什么”。
- [ ] 我知道怎么回退一次提交。

版本与时效

本文基于 Git 2.55。Git 命令长期稳定，GitHub 界面会变，但仓库、提交、分支、推送的概念不变。

Python 环境与依赖

适用场景

你换了电脑，项目跑不起来；同一个包在两个项目里版本冲突；队友说“我这里能跑”；`pip install` 装到一半报错。

最小做法

一个项目一个虚拟环境

不要把所有包装在全局 Python 里。

用 `venv`：

```powershell
cd D:\work\project
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

macOS/Linux：

```bash
python3 -m venv .venv
source .venv/bin/activate
```

退出：

```powershell
deactivate
```

记录依赖

```powershell
pip freeze > requirements.txt
pip install -r requirements.txt
```

规则：

- 项目根目录放 `requirements.txt` 或 `pyproject.toml`。
- 锁定版本：`numpy==2.1.3`，不要只写 `numpy`。
- 开发依赖和运行依赖分开：`requirements-dev.txt`。
- 不要把 `.venv/` 提交到 Git。

选 Python 版本

- 新项目优先 3.11/3.12；库兼容性最好。
- 老项目按 `README` 指定版本，不要硬升。
- Windows 用官方安装包或 `winget`；不要装多个来路不明的 Python。
- 用 `py -0p` 查看已安装版本。

常见安装问题

- 公司/学校网络慢：换镜像源。
- 需要编译的包（如某些科学计算库）：优先用 `conda` 或预编译 wheel。
- 权限错误：不要用管理员全局安装，回到虚拟环境。
- 包冲突：新建干净环境重装，不要在一个环境里反复升降级。

进阶做法

- 用 `uv` 管理环境和依赖，速度快：

```powershell
uv venv
uv pip install -r requirements.txt
```

- 用 `conda` 管理需要 C/Fortran 依赖的科学计算环境：

```powershell
conda create -n project python=3.11
conda activate project
```

- 用 `pyproject.toml` 记录项目元数据和依赖。
- 用 `pip-tools` 生成锁定文件。
- 在 README 写清：Python 版本、安装命令、运行命令、已知问题。

常见坑

- 全局装包，项目之间互相污染。
- 只写 `pip install xxx`，不记录版本，半年后复现不了。
- 把 `.venv/` 提交进 Git，仓库巨大。
- 在系统 Python 里 `pip install`，把系统工具搞坏。
- 混用 conda 和 pip，依赖冲突难排查。
- 不写 README，队友不知道用哪个 Python。
- 安装失败就 `--force-reinstall`，越修越乱。

自检清单

- [ ] 每个项目有独立虚拟环境。
- [ ] `.venv/` 在 `.gitignore` 里。
- [ ] 依赖版本已锁定。
- [ ] README 写了 Python 版本和安装命令。
- [ ] 运行命令能一条命令复现。
- [ ] 新环境从零安装能成功。
- [ ] 没有把包全局安装到系统 Python。

版本与时效

本文基于 Python 3.11、pip 26、uv 0.8、conda 23。命令会变，但“一项目一环境 + 锁版本 + README 可复现”的原则不变。

GitHub 私有仓库与协作

适用场景

你要把项目代码/文档放到 GitHub，暂时不想公开；队友要一起改；你要留版本、发提交包、做自动化检查。

最小做法

建私有仓库

1. GitHub → New repository。
2. 选 **Private**。
3. 不要勾“Add a README”，避免和本地冲突。
4. 本地：

```powershell
git init
git add .
git commit -m "chore: initial commit"
git branch -M main
git remote add origin https://github.com/<user>/<repo>.git
git push -u origin main
```

分支

- `main` 保持可交付。
- 每个功能/修复开一个分支：`feat/login`、`fix/typo`。
- 改完提 PR，合并后删分支。

```powershell
git switch -c feat/export
git add .
git commit -m "feat: add pdf export"
git push -u origin feat/export
```

Issue

- 一个 issue 只做一件事。
- 写清：背景、期望结果、验收标准。
- 用 label 区分 `bug`、`docs`、`enhancement`。
- 关闭 issue 时写一句“怎么验证的”。

PR

- 标题写“做了什么”。
- 描述写：改了什么、为什么、怎么验证。
- 小步提交，一次 PR 不要改 50 个文件。
- 合并前至少自己跑一遍测试/构建。

Release

- 打 tag：`v1.0.0`。
- Release 附：变更说明、安装包/PDF、`SHA256SUMS`。
- 提交包和源码分开，不要把大文件塞进 Git。

Actions 入门

最小 CI：

```yaml
name: CI
on: [push, pull_request]
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - run: pip install -r requirements.txt
      - run: python -m unittest discover -s tests
```

进阶做法

- 用 `CODEOWNERS` 指定审查人。
- 用分支保护：禁止 force push、要求 PR、要求 CI 通过。
- 用 Dependabot 自动更新依赖。
- 用 Issue 模板/PR 模板统一格式。
- 用 `git tag -s` 签名发布。
- 用 `gh` CLI 少点网页。

常见坑

- 私有仓库里放密钥、密码、个人信息，转公开后全泄露。
- 把数据集、模型权重、安装包提交进 Git，仓库几个 GB。
- 直接在 `main` 上改，出错无法回退。
- PR 太大，队友没法 review。
- CI 权限过大，PR 能改工作流/读 secrets。

- Release 没有校验文件，下载后不知道是否完整。
- 仓库公开前没清理 Git 历史。

自检清单

- [ ] 仓库是 Private（公开前）。
- [ ] `.gitignore` 排除依赖、数据、密钥。
- [ ] `main` 可交付，功能在分支上开发。
- [ ] PR 描述写清“改了什么/为什么/怎么验证”。
- [ ] CI 只读权限，action 固定版本。
- [ ] Release 附变更说明和 SHA256SUMS。
- [ ] 公开前扫描 Git 历史中的敏感信息。

版本与时效

本文基于 Git 2.55、GitHub 2026 界面、GitHub Actions。界面会变，但“分支 + PR + CI + Release”的协作模型不变。

数据清洗与可复现

适用场景

你跑出一个结果，过两天自己都复现不了；队友用同一份数据跑出不同结果；答辩时被问“这个数怎么来的”。

最小做法

原始数据只读

```text
data/
├─ raw/          # 原始数据，永不修改
├─ interim/      # 清洗中间结果
└─ processed/    # 最终分析数据
```

- `raw` 只读，文件名带日期和来源。
- 清洗脚本放 `src/`，每一步可重跑。
- 处理结果写 `processed`，不覆盖原始数据。

清洗步骤写进脚本

不要在 Excel 里手动改 20 步，然后说“数据已经清洗好了”。把步骤写成代码：

```python
import pandas as pd
from pathlib import Path

raw = pd.read_csv("data/raw/20261017_measure.csv", encoding="utf-8")
df = (
    raw.rename(columns={"Time(s)": "time_s", "Value(mm)": "value_mm"})
       .dropna(subset=["time_s", "value_mm"])
       .query("0 ≤ value_mm ≤ 100")
       .sort_values("time_s")
)
Path("data/processed").mkdir(parents=True, exist_ok=True)
df.to_csv("data/processed/clean.csv", index=False)
```

每一步都写注释：为什么删、为什么截断、阈值从哪来。

固定随机种子

```python
import random
import numpy as np

random.seed(42)
np.random.seed(42)
```

深度学习还要设置框架种子；能固定就固定，不能固定的写清原因。

记录环境

- Python 版本、依赖版本。
- 操作系统、GPU/CPU。
- 数据文件哈希。
- 运行命令。

`复现清单.json` 示例：

```json
{
  "python": "3.11.9",
  "command": "python src/train.py --config configs/base.yaml",
  "data_sha256": "…",
  "seed": 42,
  "result": {"rmse": 0.97}
}
```

进阶做法

- 用 `pytest` 给清洗函数写测试：空值、越界、重复、类型错误。
- 用 `pydantic` / `pandera` 做数据校验。
- 用 `Makefile` / `just` / `invoke` 把“一条命令复现”固化。
- 用 `DVC` 或 Git LFS 管理大数据版本。
- 每次实验建一个 `runs/日期_配置/`，存日志、指标、图。

常见坑

- 手动改 Excel，原始数据被覆盖。
- 清洗规则只在脑子里，换人就跑偏。
- 删除缺失值时没记录删了多少、为什么。
- 随机种子没固定，结果每次不同。
- 只记录最终指标，不记录中间步骤。
- 环境和依赖没锁，换机器跑不起来。
- 图是截图，数据表找不到对应源。

自检清单

- [ ] 原始数据只读，未修改。

- [ ] 清洗步骤在脚本里，可重跑。
- [ ] 每一步有注释和依据。
- [ ] 随机种子固定。
- [ ] 环境、依赖、命令、数据哈希有记录。
- [ ] 一条命令能从 raw 跑到 processed。
- [ ] 关键结果有对应数据表和图。

版本与时效

本文基于 Python 3.11、pandas 2.x。工具会变，但“原始数据只读 + 脚本化清洗 + 固定种子 + 复现清单”的原则不变。

可视化与出图

适用场景

你要把实验结果放进论文/答辩/报告；图被说“看不清”“配色太花”“没有误差棒”；导出后字体模糊。

最小做法

先定结论，再选图

- 比较类别：柱状图。
- 趋势：折线图。
- 分布：直方图/箱线图。
- 两个变量关系：散点图。
- 占比：饼图（类别少）。
- 多指标对比：雷达图慎用，容易误导。

一张图只表达一个结论。标题写结论，不写“结果图”。

坐标轴与图注

- 横纵轴都有标题和单位。
- 刻度从 0 开始（除非有充分理由截断）。
- 图例不遮挡数据。
- 图注写清：样本量、条件、误差棒含义、显著性标记。
- 颜色盲友好：避免红绿对比，用蓝/橙、紫/黄。

Matplotlib 最小模板

```python
import matplotlib.pyplot as plt
import numpy as np

x = np.array([1, 2, 3, 4])
y = np.array([2.1, 3.9, 6.2, 8.1])
err = np.array([0.2, 0.3, 0.2, 0.4])

fig, ax = plt.subplots(figsize=(6, 4), dpi=300)
ax.errorbar(x, y, yerr=err, fmt="o-", capsize=4, color="#2563eb")
ax.set_xlabel("时间 / s")
ax.set_ylabel("位移 / mm")
ax.set_title("位移随时间线性增加")
ax.grid(alpha=0.3)
fig.tight_layout()
fig.savefig("figure.png", dpi=300, bbox_inches="tight")
```

导出

- 论文/报告优先矢量图：PDF/SVG。
- 位图至少 300 DPI，宽度按版面定。
- 导出前检查：字体嵌入、中文不乱码、线宽、图例。
- 不要用截图；截图无法放大。

进阶做法

- 用统一色板和字号，保证多张图风格一致。
- 用 `seaborn` 快速画统计图，但不要默认主题直接用。
- 用 `SciencePlots` 或期刊模板调整样式。
- 多面板图用 `subplots`，标 `(a)(b)(c)`。
- 误差棒写清是标准差、标准误还是置信区间。
- 颜色之外再加线型/标记，黑白打印也能区分。

常见坑

- 3D 柱状图、渐变、阴影，读者看不清数值。
- 饼图类别太多，角度无法比较。
- 坐标轴截断，夸大差异。
- 没有误差棒/样本量，结论不可信。
- 图注只写“结果如图”，没有条件说明。
- 中文乱码：没设置中文字体。
- 导出 PNG 72 DPI，打印模糊。
- 颜色红绿搭配，色盲读者无法区分。

自检清单

- [ ] 一张图只表达一个结论。
- [ ] 坐标轴有标题和单位。
- [ ] 图例不遮挡数据。
- [ ] 图注写清样本量、条件、误差棒含义。
- [ ] 颜色盲友好，黑白打印可区分。
- [ ] 导出矢量图或 300 DPI 位图。
- [ ] 中文、字体、线宽检查过。

版本与时效

本文基于 Matplotlib 3.10、Python 3.11。绘图库版本会变，但“先结论后图 + 坐标轴/图注完整 + 矢量导出”的原则不变。

VS Code 配置

适用场景

你刚装 VS Code，插件一堆但不知道用哪个；打开项目全是红波浪线；想连服务器/另一台电脑写代码。

最小做法

先打开“文件夹”，不要只开单文件

- 文件 → 打开文件夹，选项目根目录。
- 项目根目录放 `.vscode/settings.json`，团队共用配置。
- 个人偏好放用户设置，不要提交到仓库。

插件最小集

按需安装，不要一次装 50 个：

- Python + Pylance：Python 补全/调试。
- C/C++：C++ 补全/调试。
- GitLens：看提交历史。
- Markdown All in One：目录、快捷键。
- markdownlint：Markdown 格式检查。
- Mermaid Preview：预览 Mermaid。
- Draw.io Integration：在 VS Code 里画图。

前端/嵌入式/LaTeX 再按项目加。

工作区配置

`.vscode/settings.json`：

```json
{
  "files.encoding": "utf8",
  "files.eol": "\n",
  "editor.formatOnSave": true,
  "editor.rulers": [88],
  "python.defaultInterpreterPath": ".venv\\Scripts\\python.exe",
  "python.terminal.activateEnvironment": true,
  "markdown.preview.breaks": true
}
```

Python 解释器

1. `Ctrl+Shift+P` → Python: Select Interpreter。
2. 选项目 `.venv` 里的 Python。
3. 终端激活环境后再装包。
4. 左下角确认解释器路径。

远程开发

- 装 Remote - SSH 插件。
- `Ctrl+Shift+P` → Remote-SSH: Connect to Host。
- 在服务器上装 Python/扩展（部分扩展需远程安装）。
- 远程终端里跑命令，文件在服务器上。

进阶做法

- 用 Profiles 区分“Python” “嵌入式” “写作”三套配置。
- 用 Settings Sync 同步个人设置，但不要把密钥同步上去。
- 用 `.editorconfig` 统一缩进/换行。

- 用任务（tasks.json）一键构建/测试。
- 用调试配置（launch.json）复现运行环境。
- 用 Dev Container 固定开发环境。

常见坑

- 只打开单文件，Python 导入路径全错。
- 解释器选到全局 Python，包装错地方。
- 插件太多，启动慢、冲突多。
- 把 `.vscode/` 里的本机绝对路径提交到仓库。
- 远程开发时扩展装在本地，服务器上用不了。
- 自动格式化把整个文件重排，diff 爆炸。
- 设置同步把 token/密钥传到云端。

自检清单

- [ ] 打开的是项目文件夹，不是单文件。
- [ ] 解释器指向项目 `.venv`。
- [ ] `.vscode/settings.json` 不含本机绝对路径/密钥。
- [ ] 插件按需安装，没有明显冲突。
- [ ] 保存时格式化不会重排整个文件。
- [ ] 远程开发扩展装在远程端。
- [ ] 设置同步没有包含敏感信息。

版本与时效

本文基于 VS Code 1.141。插件名称和行为会变，但“工作区 + 解释器 + 最小插件集”的原则不变。

代码质量与测试

适用场景

代码只有你能看懂；改一个函数坏三个地方；队友接手要重写；答辩现场演示崩了。

最小做法

结构

```text
project/
├─ src/              # 源码
├─ tests/            # 测试
├─ data/             # 数据（raw/processed）
├─ configs/          # 配置
├─ scripts/          # 一次性脚本
└─ README.md
```

- 一个文件只做一类事。
- 函数短小，输入输出明确。
- 配置和代码分离，不要把路径/密钥写死。
- 入口文件只做参数解析和调用，不写业务逻辑。

命名

- 变量名说清含义：`motor_speed` 而不是 `a`。
- 函数名用动词：`load_data`、`compute_rmse`。
- 常量全大写：`MAX_RETRY`。
- 避免拼音缩写和无关梗。

注释

- 注释写“为什么”，不写“是什么”。
- 复杂公式写来源和假设。
- TODO 写清“谁、什么时候、做什么”。
- 不要用注释屏蔽大段死代码，直接删（Git 有历史）。

日志

```python
import logging

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s
logging.info("loaded %d rows", len(df))
logging.warning("missing values: %d", df.isna().sum().sum())
```

- 关键步骤有日志。
- 错误日志带上下文，不要只打“failed”。
- 不要用 `print` 做正式日志。

单元测试

```python
import unittest
from src.metrics import rmse

class TestMetrics(unittest.TestCase):
    def test_rmse_zero(self):
        self.assertEqual(rmse([1, 2], [1, 2]), 0)

    def test_rmse_positive(self):
        self.assertAlmostEqual(rmse([1, 2], [2, 4]), 1.5811, places=3)
```

- 每个核心函数至少一个正常用例 + 一个边界用例。
- 测试不依赖本机路径和网络。
- 提交前跑一遍。

进阶做法

- 用 `pytest` 替代 `unittest`，参数化测试。
- 用 `ruff` / `flake8` 做静态检查，`black` / `ruff format` 统一格式。
- 用 `mypy` 做类型检查。
- 用 CI 跑测试，PR 不通过不合并。
- 用 `coverage` 看覆盖，但不要追求 100%。
- 用 `pre-commit` 在提交前自动检查。

常见坑

- 一个文件几千行，函数几百行。
- 变量名 `a`、`b`、`tmp`，一周后自己看不懂。
- 注释写“这里加 1”，没说为什么。
- 测试只测“能跑”，不测边界。
- 日志打一堆，关键错误没上下文。
- 代码里写死本机路径/密钥。
- 没有 README，队友不知道从哪运行。

自检清单

- [ ] 目录结构清晰，源码/测试/数据分开。
- [ ] 函数短小，命名能看懂。
- [ ] 注释解释“为什么”。
- [ ] 关键步骤有日志。
- [ ] 核心函数有测试，含边界用例。
- [ ] 提交前跑测试/静态检查。
- [ ] README 写清安装、运行、测试命令。

版本与时效

本文基于 Python 3.11、pytest 8、ruff 0.8。工具会变，但“结构清晰 + 命名可读 + 测试覆盖核心逻辑”的原则不变。

AI 辅助编程边界

适用场景

你想用 AI 写代码/解释报错/生成测试；担心代码有漏洞、依赖不存在、竞赛规则不允许；不知道怎么声明。

最小做法

适合交给 AI 的事

- 解释报错和概念。
- 生成样板代码（读文件、画图、写测试骨架）。
- 重构小函数、补类型标注。
- 把一段代码翻译成另一种写法。
- 生成正则/命令/配置的初稿。

不适合直接交给 AI 的事

- 未经审查的完整项目。
- 涉及账号、密钥、隐私的代码。
- 竞赛/课程明确禁止 AI 生成的核心内容。
- 你不能解释、不能调试的代码。
- 需要现场演示、必须稳定的关键路径。

审查四步

1. **能运行**：在干净环境跑通。
2. **能解释**：每一行你能说清为什么。

3. **能测试**：核心逻辑有测试/边界用例。
4. **能复现**：依赖、版本、命令写进 README。

AI 代码最常见的坑：

- 引用不存在的库/函数/参数。
- 版本不兼容。
- 异常处理缺失。
- 安全漏洞（命令注入、路径穿越、明文密钥）。
- 只处理理想输入，边界全崩。

声明

按竞赛/课程要求声明 AI 使用范围。没有模板时，至少写清：

- 工具与版本。
- 用在哪个环节（解释/生成/重构/测试）。
- 哪些内容由你核对/修改。
- AI 生成内容不替代你的判断。

进阶做法

- 给 AI 的上下文：目标、输入输出、约束、已有代码、错误日志。
- 一次只让它改一个函数/一个文件。
- 要求它先给方案，再给代码。
- 让它生成测试，但你要检查测试是否真的覆盖边界。
- 用 `git diff` 审查每一行改动，不要整段粘贴。
- 重要模块让 AI 做“审查者”，你来做决定。

常见坑

- 直接粘贴 AI 代码，不运行、不理解。

- AI 编造依赖/API，安装时报错。
- 把密钥/隐私贴进对话。
- 用 AI 生成竞赛核心内容，违反规则。
- 代码能跑但不可维护，队友看不懂。
- 不声明 AI 使用，答辩被问住。
- 让 AI 一次改十个文件，diff 无法审查。

自检清单

- [ ] 我知道哪些环节用了 AI。
- [ ] 代码在干净环境跑通。
- [ ] 每一行我能解释。
- [ ] 核心逻辑有测试/边界用例。
- [ ] 依赖/版本/命令写进 README。
- [ ] 没有把密钥/隐私贴进对话。
- [ ] 按竞赛/课程要求做了声明。

版本与时效

本文不绑定具体模型。AI 工具和规则会变，但“人工审查 + 可运行 + 可解释 + 合规声明”的边界不变。

卷四 · 项目推进力

这一卷不告诉你“做什么项目”，因为每年赛道、政策、评审口味都在变。它只给判断框架：

- 这项竞赛值不值得投入。
- 这个题目能不能在时间和能力范围内做完。
- 队友怎么分工，失败了怎么退出。
- 申报书、答辩、复盘分别该准备什么。

本卷章节

- 竞赛值不值得参加：认可度、能力匹配、时间成本、复用性。
- 选题：从问题出发：题目来源、筛选标准、最小可行题目。
- 组队与分工：角色、交付物、退出机制、沟通节奏。
- 读规则与评分表：提交物清单、评分表反推、时间线。
- 项目管理与周报：里程碑、任务板、周报、风险登记。
- 申报书写作：结构、证据链、常见扣分点。
- 原型与实验：最小闭环、迭代、测试、成本。
- 答辩准备：PPT 结构、讲稿、问答、演示风险。
- 复盘方法：目标 vs 结果、决策回顾、可复用资产。
- 学业平衡：时间预算、优先级、止损。

先读哪篇

如果你正在犹豫要不要报名，先读竞赛值不值得参加。如果你已经组队，直接跳到“组队”和“项目管理”。

竞赛值不值得参加

适用场景

新学期开始，群里转发了一堆竞赛通知；或者你已经报了一个比赛，但不确定要不要继续投入。你需要一个判断框架，而不是一句“多参加总没坏处”。

最小做法

用四个维度打分，每个维度 1-5 分，总分不作为绝对标准，但能逼你把问题想清楚。

维度一：认可度

- 学校/学院的竞赛名录里有没有它?
- 保研、评奖、学分认定是否认可？以当年文件为准。
- 赛事本身是否有稳定的评审机制和公开规则?

查证方法：学校教务处/学院官网 → 当年的竞赛认定文件；竞赛官网 → 主办方、章程、往届评审。不要只看培训机构的宣传。

维度二：能力匹配

- 你需要补多少新技能？两周能补完，还是两个月?
- 队伍里有没有人能覆盖关键环节（机械、电控、算法、写作、答辩）？
- 你在这个项目里具体负责什么？如果说不清，大概率会变成“打杂”。

维度三：时间成本

粗算一下：

```text
总投入 ≈ 备赛周数 × 每周小时数 + 冲刺期额外小时
```

再对照你的课程、考试、科研、其他竞赛。如果总投入超过你能稳定拿出的时间，结果通常是“报名了，但没做完”。

维度四：复用性

一次投入能不能沉淀成至少两样东西：

- 一篇论文/专利/软著；
- 毕业设计的一部分；
- 下一场比赛的原型；
- 一项能写进简历的硬技能；
- 一套可复用的代码/文档/流程。

只能拿一个名次、不能沉淀的竞赛，性价比通常不高。

进阶做法

组队前问三个问题

1. **谁负责什么？** 具体到模块和交付物，不写“大家一起做”。
2. **失败了怎么办？** 是继续迭代、转投其他比赛，还是及时止损？
3. **退出机制是什么？** 有人中途退出，工作怎么交接，署名怎么处理？

这三个问题不问清楚，后期大概率会出矛盾。

把“不参加”当成一个正式决策

不参加不是懒。把时间留给绩点、科研、英语、实习，可能回报更高。关键是想清楚：

- 你放弃的是什么？
- 你换来了什么？
- 这个交换在一年后还成立吗？

用“最小可交付”反推

先问：这个比赛最少要交什么？然后倒推：

- 报名截止：
- 初稿截止：
- 原型完成：
- 测试完成：
- 答辩准备：

每个节点写一个负责人和一个可验收的产物。

常见坑

- 看别人报就报，最后队伍里没人真正负责。
- 只算比赛那几天，不算前期准备和后期返工。
- 选了一个和自身能力完全不匹配的赛道，靠熬夜硬撑。
- 为了“加分”参加多个比赛，每个都只做了一半。
- 不查当年政策，拿去年的认定文件做决定。
- 组队靠关系，不分工，最后一个人扛全部。

自检清单

- [ ] 我查过学校/学院当年的竞赛认定文件。
- [ ] 我能说清这个比赛要交什么、什么时候交。
- [ ] 我算过总时间投入，并和课程/考试对照过。
- [ ] 我知道自己具体负责哪个模块。
- [ ] 队伍里关键环节都有人覆盖。
- [ ] 我们讨论过失败和退出机制。
- [ ] 这个项目至少能沉淀出两样可复用的东西。

版本与时效

本文只给判断框架，不评价任何具体赛事的含金量。学校认定政策、赛事规则每年都可能变化，务必以当年官方文件为准。

选题：从问题出发

适用场景

你要参加竞赛、做课程项目、申报大创/SRTP，脑子里有一堆技术名词，但不知道做什么；或者题目太大，做到一半发现做不完。

最小做法

先找问题，不先找技术

题目来源：

- 课程/实验里让你反复手动处理的环节。
- 实验室/课题组已经在做、但缺一块的小问题。
- 校园/社区/家里真实存在的麻烦。
- 竞赛官方榜单/企业命题（如果有）。
- 你熟悉的软件/设备里明显不顺手的地方。

技术是手段，不是题目。先说清“谁在什么场景下遇到什么问题”，再谈用什么技术。

选题四问

1. **谁受影响？** 具体到人/角色，不要写“社会”。
2. **现有方案缺什么？** 贵、慢、不准、难用、不可靠，选一个主要矛盾。
3. **你能做什么？** 在你的时间、人手、设备范围内，能交付哪一块。
4. **怎么证明有效？** 一个可测指标：时间、成本、误差、成功率、覆盖率。

四问答不上来，题目还没定型。

最小可行题目

把题目压到一句话：

> 为____（谁），在____（场景）下，做一个____（交付物），把____（指标）从____做到____。

例子（只示范结构，不指定方向）：

> 为实验室同学，在每周数据整理时，做一个自动汇总脚本，把 2 小时手工整理降到 10 分钟。

题目越小，越容易做完；做完再扩，不要一开始就写“平台”。

进阶做法

- 做一张“问题—证据—方案—指标”表，逐项填。
- 找 3 个潜在用户聊 10 分钟，确认问题真实存在。
- 查现有方案/论文/专利，确认不是重复造轮子。
- 把题目拆成 3 个里程碑，每个里程碑有可演示的交付物。
- 预留一个“保底版本”：即使核心创新失败，也能交付一个完整的基础版。

常见坑

- 题目写成“基于 XX 技术的 XX 平台”，没有具体问题。
- 技术堆砌：AI+区块链+数字孪生，但没有一个能验收。
- 伪需求：自己觉得酷，实际没人用。
- 题目太大：三个人做半年也做不完。
- 没有指标，答辩时说不清“做成了什么”。
- 忽略合规/隐私/安全，做到一半发现不能公开。

自检清单

- [ ] 我能用一句话说清“谁、什么场景、什么问题”。
- [ ] 我找到至少一个现有方案的不足。
- [ ] 我知道自己具体交付哪一块。
- [ ] 我有一个可测指标。
- [ ] 题目能在团队时间和能力范围内完成。
- [ ] 我查过现有方案，不是重复造轮子。
- [ ] 我有保底版本。

版本与时效

本文只给选题框架，不指定具体方向。每年赛道、政策、评审口味都在变，选题要结合当年规则和自身条件。

组队与分工

适用场景

你要组队参加竞赛/做项目；队友很熟但没人干活；分工写成“大家一起做”；有人中途消失；署名和贡献说不清。

最小做法

先定角色，不先定人头

常见角色：

- **队长/协调**：定节奏、对外沟通、推进决策。
- **技术**：核心方案、代码/结构/实验。
- **文档**：申报书、报告、图表、提交包。
- **答辩**：PPT、讲稿、问答、现场演示。
- **后勤**：材料、采购、场地、报销、时间提醒。

一个人可以兼角色，但每个角色要有人负责。

分工到“交付物”

不要写“张三负责技术”，要写：

成员	负责模块	交付物	截止时间	备份人
A	结构设计	图纸 v1 + BOM	10-30	B
B	控制	可运行代码 + 测试记录	11-10	A
C	文档	申报书 v1 + 图表	11-15	D

交付物要可验收：文件、演示、测试结果，不是“参与讨论”。

退出机制

组队第一天就谈：

- 有人退出，工作怎么交接？
- 已经做的工作怎么署名？
- 退出后还能不能使用共同成果？
- 什么情况下算“自动退出”（连续两周不响应）？

写进一份简单的组队约定，大家确认。

沟通节奏

- 每周一次 30 分钟例会：上周完成、本周计划、阻塞。
- 日常用群/文档异步沟通，重要决定留文字。
- 文档/代码集中在一个地方，不要散在聊天记录里。
- 截止前两周加一次“每日 10 分钟站会”。

进阶做法

- 用 GitHub Projects/飞书多维表格做任务板。
- 每周发周报，抄送队友/指导老师。
- 关键决策写“决策记录”：背景、选项、选择、理由。
- 设一个“红队”角色，专门挑方案漏洞。
- 给每个模块设备份人，避免单点故障。

常见坑

- 靠关系组队，不分工，最后一个人扛全部。
- 分工写“大家一起做”，没人负责。
- 不谈心退出，期末/赛前有人消失。
- 所有沟通在群里，文件过期/聊天记录找不到。
- 队长一个人做所有决定，其他人躺平。
- 署名按“谁嗓门大”，不按实际贡献。
- 指导老师直到提交前才看到东西。

自检清单

- [ ] 每个角色有人负责。
- [ ] 分工写到交付物和截止时间。
- [ ] 有备份人。
- [ ] 讨论过退出机制和署名规则。
- [ ] 每周有例会/周报。
- [ ] 重要决定有文字记录。
- [ ] 指导老师知道进度和风险。

版本与时效

本文的团队框架不依赖具体赛事。不同竞赛对队员人数/角色有规定，以当年规则为准。

读规则与评分表

适用场景

你拿到一份几十页的竞赛通知/申报指南，不知道重点在哪；提交前才发现漏了材料；答辩被问“你们这个不符合要求”。

最小做法

先做三张表

1. 提交物清单

提交物	格式	页数/大小	截止时间	负责人	状态
申报书	PDF	≤20 页	11-30	A	未开始
演示视频	MP4	≤3 分钟	11-30	B	未开始
源代码	ZIP	≤100 MB	11-30	C	未开始

2. 评分表

把评分项抄下来，逐项问：我们哪一页/哪个证据对应它？

评分项	分值	我们的证据	页码
创新性	30	方案对比 + 专利检索	5-7
可行性	25	原型 + 测试数据	8-10
团队协作	15	分工表 + 周报	11

3. 时间线

- 报名截止。
- 材料提交截止。
- 初赛/复赛/决赛。
- 答辩时间。
- 每个节点前倒推 3 天作为内部截止。

读“资格”和“格式”

- 参赛资格：年级、专业、人数、指导教师、是否允许跨校。
- 材料格式：模板、字体、页数、文件命名、压缩包结构。
- 原创/查重/AI 使用要求。
- 知识产权/保密要求。

不满足资格，后面做得再好也没用。

读“评审流程”

- 网评/会评/答辩各占多少?
- 评委是谁（专家/企业/投资方）?
- 每一轮看什么材料?
- 有没有现场演示/实物?

不同轮次准备不同版本的材料，不要一份 PPT 打天下。

进阶做法

- 用荧光笔标出“必须”“不得”“建议”。
- 把评分表贴在工作区，每周对照。
- 找往年获奖作品/评审标准（公开的），看结构。
- 给每个评分项指定一个“证据负责人”。
- 提交前做一次“规则逐条核对”。

常见坑

- 只看截止时间，不看材料格式。
- 申报书超页数，被直接筛掉。
- 评分表里的“团队协作”没证据。
- 忽略“必须提交原创声明/查重报告”。
- 不同轮次用同一份材料，重点不对。
- 文件名/压缩包结构不符合要求，系统上传失败。
- 截止前 1 小时才上传，系统崩溃。

自检清单

- [ ] 我做了提交物清单，逐项有负责人。
- [ ] 我抄了评分表，每项有对应证据。
- [ ] 我确认了参赛资格。
- [ ] 我确认了格式/页数/命名要求。
- [ ] 我知道每一轮评审看什么。
- [ ] 内部截止比官方截止早 3 天。
- [ ] 提交前逐条核对过规则。

版本与时效

本文只给读规则的方法。具体规则以当年官方通知/指南为准；不同赛事差异很大，不要套用上一届。

项目管理与周报

适用场景

项目一开始很热闹，两周后没人提；任务都在脑子里；截止前发现关键模块没做；指导老师问进度，你只能现编。

最小做法

先定里程碑

里程碑不是“做完”，而是“能演示/能提交”的节点：

里程碑	日期	交付物	验收标准
M1 方案冻结	10-20	方案文档 + 风险表	关键选型有依据
M2 原型跑通	11-10	可运行原型 + 测试记录	主流程能演示
M3 材料齐套	11-25	申报书 + 视频 + 代码	按提交清单逐项核对
M4 答辩准备	12-01	PPT + 讲稿 + 问答	完整试讲两遍

每个里程碑指定负责人和备份人。

任务板

三列就够：

- **待做**：本周要做的。
- **进行中**：最多 3 项，多了就是没聚焦。
- **已完成**：附交付物链接。

工具：GitHub Projects、飞书多维表格、Trello、Excel 都行。关键不是工具，是每周更新。

周报

每周固定时间发，模板：

```text
本周完成：
- ...
证据：文件/链接/截图

阻塞：
- ...（需要谁、什么时候）

下周计划：
- ...

风险：
- ...
```

周报不是流水账，写“结果 + 证据 + 阻塞”。

风险登记

风险	触发条件	影响	应对	负责人
关键模块延期	11-05 仍未跑通	无法演示	启用保底方案	A
队员退出	连续两周无响应	人手不足	备份人接手	B
设备故障	测试中断	数据缺失	备用设备/延期申请	C

每周更新一次，不要等出事才想。

文档留痕

- 会议记录：日期、参与人、决定、待办。
- 决策记录：背景、选项、选择、理由。

- 版本记录：谁在什么时候改了什么。
- 提交记录：提交时间、版本、回执。

进阶做法

- 用 Issue 管理任务，一个 issue 一个交付物。
- 用标签区分 `blocked`、`review`、`done`。
- 每周看一次“燃尽图/完成率”，但不迷信。
- 给关键路径留 20% 缓冲。
- 指导老师每两周同步一次，不要消失。

常见坑

- 里程碑写成“完成方案”，没有验收标准。
- 任务板建了不更新，变成摆设。
- 周报只写“在做了”，没有证据。
- 风险表只列“时间紧”，没有应对。
- 所有决定在群里，过两周找不到。
- 关键路径没有备份人。
- 截止前才发现依赖外部设备/场地。

自检清单

- [ ] 有 3-4 个可验收的里程碑。
- [ ] 任务板每周更新。
- [ ] 周报有结果、证据、阻塞。
- [ ] 风险表每周更新，有应对和负责人。
- [ ] 会议/决策有文字记录。
- [ ] 关键路径有缓冲和备份人。

- [ ] 指导老师定期同步。

版本与时效

本文的项目管理方法不依赖具体工具。竞赛周期、评审节点以当年规则为准。

申报书写作

适用场景

你要写竞赛申报书、大创申请书、项目结题报告；内容很多，但不知道评委想看什么；写出来像说明书，没有重点。

最小做法

先写四块，再扩全文

1. **一句话摘要**：为谁、解决什么、做到什么程度。
2. **目标与指标**：可验证，不写“提升用户体验”。
3. **技术路线**：总体图 + 3-4 个关键模块。
4. **证据**：原型、测试、对比、用户反馈。

这四块写不清，后面扩写都是空话。

推荐结构

1. 项目摘要
2. 背景与问题（谁受影响、现有方案缺什么）
3. 目标与指标（量化、验收方式）
4. 方案与技术路线（总体图 + 模块）
5. 创新点（和现有方案比，具体改了什么）
6. 可行性（技术、时间、人员、经费）
7. 进度安排（里程碑 + 负责人 + 交付物）
8. 风险与应对（至少三条）
9. 经费预算（科目、单价、数量、用途）

10. 附录（图纸、代码说明、数据、参考文献）

证据链

每个结论后面跟证据：

- “效率提升 40%” → 测试条件、样本量、对比基线。
- “成本降低 30%” → BOM、报价、计算过程。
- “创新” → 专利/论文检索、方案对比表。
- “可行” → 原型照片、测试记录、团队成员能力。

没有证据的形容词，删掉或改成“设计目标”。

图与表

- 总体路线图：一页看懂。
- 模块图：接口清楚。
- 进度甘特图：里程碑和负责人。
- 对比表：现有方案 vs 你的方案。
- 测试数据表：条件、样本、指标、误差。

图注写清“图几、标题、数据来源”。

进阶做法

- 先写评分表对应的章节，再补过渡。
- 摘要最后写，控制在 300 字内。
- 每一节开头先给结论，再展开。
- 用“问题—方案—证据—边界”四段写每个创新点。
- 找非本专业的人读一遍，看能否看懂。
- 提交前做“陌生人视角”审查：假设你不认识作者，能否看懂。

常见坑

- 背景写三页，问题一句话带过。
- 创新点写“用了 AI/数字孪生”，没有对比。
- 指标不可验证：“显著提升”“大幅降低”。
- 技术路线图像流程图，没有接口和数据流。
- 预算写“其他”，没有明细。
- 风险和应对是“加强管理”“及时沟通”。
- 附录塞满，正文没有结论。
- 文档属性带真实姓名/单位，公开前没清理。

自检清单

- [ ] 摘要能一句话说清“谁、什么问题、做到什么程度”。
- [ ] 每个目标有可验证指标。
- [ ] 技术路线图一页看懂。
- [ ] 每个创新点有对比和证据。
- [ ] 进度表有负责人和交付物。
- [ ] 风险至少三条，有触发条件和应对。
- [ ] 预算有单价、数量、用途。
- [ ] 非本专业的人能读懂。

版本与时效

本文只给写作框架，不提供可直接提交的申报书。不同赛事/学校模板不同，以当年官方模板为准。

原型与实验

适用场景

你要把方案做成能演示的东西；一上来就想做完整系统，结果截止前只完成一半；测试记录缺失，答辩说不清数据。

最小做法

先做“能演示的最小闭环”

最小闭环 = 输入 → 处理 → 输出 → 可见结果。

- 机械：一个机构能完成一个动作。
- 电控：一个传感器能读数，一个执行器能动作。
- 软件：一条命令能跑通主流程。
- 系统：端到端能走一遍，哪怕很粗糙。

先让闭环跑通，再谈性能、外观、扩展。

迭代节奏

- 一周一个可演示版本。
- 每个版本记录：改了什么、为什么、结果如何。
- 版本号 `v0.1`、`v0.2`，不要“最终版”。
- 每次迭代保留上一版，出问题能回退。

测试

测试类型	目的	记录
功能测试	主流程能不能走通	步骤、预期、实际
边界测试	极端输入/条件	输入、结果、是否崩溃
性能测试	速度/精度/稳定性	指标、样本量、环境
用户测试	别人能不能用	任务、耗时、反馈

测试记录要能复现：环境、版本、命令、数据。

成本

- BOM：元件、数量、单价、来源。
- 加工：工时、外协费用。
- 时间：每个人每周投入小时数。
- 迭代损耗：打样/返工次数。

成本不是只算钱，也算时间。

失败证据

保留失败的照片、日志、测试数据。答辩时主动说“这个方案试过，失败的原因是……”，比只讲成功更可信。

进阶做法

- 用“功能—成本—风险”三维评估每个方案。
- 关键模块做两个备选，主方案失败能切换。
- 用仿真/小样先验证，再做实物。
- 每次测试写“测试计划”，不要随手试。
- 用视频记录演示过程，防止现场翻车。

- 给原型加“故障恢复”步骤：断电/重启/重置。

常见坑

- 一上来做完整系统，截止前没有可演示的东西。
- 只测成功路径，边界一碰就崩。
- 测试记录只有截图，没有环境/版本。
- 原型靠一根线/一个临时补丁撑着，现场必翻车。
- 成本只算采购，不算加工和时间。
- 失败记录全删，答辩被问“你们试过什么”答不上来。
- 没有备选方案，主方案失败就结束。

自检清单

- [ ] 有一个能演示的最小闭环。
- [ ] 每周一个可演示版本。
- [ ] 每个版本有变更记录。
- [ ] 功能/边界/性能测试有记录。
- [ ] 测试环境、版本、命令可复现。
- [ ] 成本算了钱和时间。
- [ ] 保留了失败证据。
- [ ] 关键模块有备选方案。

版本与时效

本文的原型方法不依赖具体技术栈。实验安全、设备规范以实验室/学校要求为准。

答辩准备

适用场景

材料交完了，马上要答辩；PPT 很多页，不知道讲什么；现场演示怕翻车；评委一问就慌。

最小做法

先定 3 个必须记住的结论

评委听完只记得 3 件事。先写下：

1. 我们解决了什么问题。
2. 我们做到了什么程度（指标）。
3. 我们的证据是什么。

所有 PPT 页都围绕这 3 件事。

PPT 结构（10-15 分钟）

1. 封面：项目名 + 一句话定位。
2. 问题：谁、什么场景、现有方案缺什么。
3. 目标：1-3 个可验证指标。
4. 方案：总体路线图 + 关键模块。
5. 证据：原型/测试/对比。
6. 分工与进度：谁做了什么，下一步。
7. 风险与边界：至少三条。
8. 结束页：结论 + 联系方式。

每页一个结论，标题写结论，不写栏目名。

讲稿

- 每页 2-4 句口语，不写逐字稿。
- 开头 30 秒说清“做了什么、结果如何”。
- 时间分配：问题 20%，方案 30%，证据 35%，收尾 15%。
- 至少完整试讲两遍，掐表。
- 准备一个 3 分钟版本，防止临时压缩时间。

问答

准备 8 个问题：

- 和现有方案比，优势在哪？
- 关键指标怎么测的？误差多大？
- 如果核心模块失败，怎么办？
- 成本/周期/可制造性如何？
- 哪些是你们做的，哪些是现成工具？
- 创新点具体在哪？有对比吗？
- 数据/样本够不够？
- 下一步怎么做？

回答：先结论，再证据，最后边界。不会就说“这一点还没验证”。

演示风险

- 现场演示准备：本地文件 + 录屏 + 备用设备。
- 不依赖网络；必须联网的，准备热点。
- 演示前重启设备，关掉通知/弹窗。
- 准备“失败话术”：如果卡住，切录屏，继续说。

进阶做法

- 用母版统一字体、颜色、页码。
- 关键页加备份页，问答时跳转。
- 用演讲者视图看备注，但不照读。
- 找同学当“假评委”提问。
- 答辩前一天去场地试投影/音响。
- 准备一份一页纸的“核心数据卡”。

常见坑

- 第一页放 20 个项目背景，评委不知道你要讲什么。
- 每页字太多，字号小于 18pt。
- 动画过多，翻页卡顿。
- 只有方案，没有证据。
- 现场演示依赖网络/实验室设备。
- 回答问题超时，挤掉后面内容。
- 结束页只有“谢谢聆听”，没有结论。

自检清单

- [ ] 我能说出 3 个必须记住的结论。
- [ ] 每页标题是一个结论。
- [ ] 一页最多一个图 + 三条要点。
- [ ] 每个结论有证据或标注“待验证”。
- [ ] 讲稿能在规定时间内讲完。
- [ ] 试讲至少两遍，掐表。
- [ ] 准备了 8 个问答。
- [ ] 演示有录屏和本地备份。

版本与时效

本文基于常见竞赛/课程答辩流程。具体时间、页数、形式以当年通知为准。

复盘方法

适用场景

项目结束/比赛结束，大家吃顿饭就散了；下次遇到同样的问题还是踩坑；说不清哪些做对了、哪些做错了。

最小做法

先对目标，不对情绪

目标	计划	实际	差异原因
完成原型	11-10	11-18	关键模块返工
提交材料	11-25	11-30	等外协加工
答辩	12-01	12-01	正常

只写事实，不写“大家都很努力”。

决策回顾

列出 3-5 个关键决策：

时间	决策	当时依据	现在看是否成立
10-10	选方案 A	成本低	成立，但精度不够
10-25	换供应商	交期快	不成立，质量不稳

复盘不是追责，是看“当时的判断依据”够不够。

做对了什么

- 哪个流程/工具真的省了时间?
- 哪个分工方式有效?
- 哪个风险提前识别了?
- 哪个沟通机制起了作用?

写成可复用的做法，不要写“团队很团结”。

做错了什么

- 哪个决定后来证明错了？为什么?
- 哪个风险没识别?
- 哪个环节重复返工?
- 哪个沟通断层导致问题?

每条写“下次怎么改”。

可复用资产

- 代码/脚本/模板。
- 测试数据/实验记录。
- 供应商/加工渠道。
- 文档结构/答辩 PPT。
- 人脉/合作方。

资产要整理到能直接拿用的程度，不是“在某个文件夹里”。

进阶做法

- 用“5 Whys”追根因，但不要无限套娃。
- 让每个成员单独写 10 分钟，再一起对。
- 复盘会控制在 60 分钟，先事实后感受。
- 输出一页“下次提前做什么”清单。

- 把复盘文档放进项目仓库，下一届能看。

常见坑

- 复盘变成甩锅/表扬大会。
- 只写“时间紧”，不写具体哪个环节紧。
- 只写感受，不写事实和证据。
- 不记录失败，下一次继续踩。
- 资产散在各人电脑里，换人就用不了。
- 复盘完没有行动项，等于没复盘。

自检清单

- [ ] 目标 vs 结果有表格，只写事实。
- [ ] 回顾了 3-5 个关键决策。
- [ ] 做对/做错各有具体条目。
- [ ] 每条错误有“下次怎么改”。
- [ ] 可复用资产整理到能直接用的程度。
- [ ] 有一页“下次提前做什么”。
- [ ] 复盘文档进了项目仓库。

版本与时效

本文的复盘框架不依赖具体项目。奖项/结果以官方公示为准，复盘重点是过程而非名次。

学业平衡

适用场景

竞赛/项目挤占课程时间；期末和比赛撞车；什么都想抓，结果每样都差一点；不知道该不该退出某个项目。

最小做法

先算时间预算

一周 168 小时，先扣掉：

- 睡眠 49 小时（7 小时/天）。
- 上课/实验 30-40 小时。
- 吃饭/通勤/杂事 20 小时。
- 剩余 60-70 小时。

再分配给：

- 作业/复习：20-25 小时。
- 竞赛/项目：15-20 小时。
- 英语/科研/其他：5-10 小时。
- 休息/社交：10 小时。

算完就知道“再加一个比赛”是否现实。

定优先级

按“不可逆性”排序：

1. 考试/绩点（影响保研/毕业）。
2. 关键竞赛节点（错过等一年）。
3. 科研/论文（长周期）。
4. 一般竞赛/活动（可替代）。

不是所有比赛都值得牺牲绩点。

设止损线

- 连续两周项目没有实质进展 → 检查方向/分工。
- 关键模块到某日期仍未跑通 → 启用保底方案。
- 时间投入超过预算 30% → 重新评估是否退出。
- 课程出现挂科风险 → 优先保课程。

止损不是失败，是把资源挪到更重要的地方。

期末/赛前撞车

- 提前 4 周看考试/截止时间，做冲突表。
- 能提前交的作业提前做。
- 能拆的模块提前冻结。
- 和队友/老师沟通，调整分工。

进阶做法

- 用日历把所有截止时间放一起，按周看。
- 每周留半天“缓冲”，处理突发。
- 用番茄钟/时间块，避免“看起来很忙”。
- 竞赛项目按“可交付”拆，不按“工时”堆。
- 每学期只保留 1-2 个主线项目，其他做减法。

常见坑

- 报了一堆比赛，每个都只做一半。
- 期末前还在改原型，考试裸考。
- 不会拒绝，队友/老师加任务就接。
- 用熬夜补时间，第二天效率更低。
- 没有止损线，沉没成本越陷越深。
- 只看竞赛，忽略英语/科研的长期价值。
- 把所有时间排满，没有缓冲。

自检清单

- [ ] 我算过每周可支配时间。
- [ ] 我知道当前项目的优先级。
- [ ] 我给每个项目设了止损线。
- [ ] 期末/赛前冲突有提前安排。
- [ ] 每周留了缓冲时间。
- [ ] 我没有同时推进超过 2 个主线项目。
- [ ] 课程出现风险时，我会优先保课程。

版本与时效

本文的时间预算因人而异。保研/评奖政策以学校当年文件为准，不要用旧经验套。

卷六 · 经验卷

这一卷写作者自己的经历，但有明确边界：

- 只讲决策、踩坑、取舍和团队协作。
- 不写具体竞赛、项目、奖项、名次。
- 不出现队友姓名、老师姓名、学院内部信息。
- 不把经历写成简历。

为什么这样写

项目内容每年都变，直接照搬别人的方案，既容易过时，也可能涉及未公开信息。真正能复用的是判断方式：

- 当时为什么选这个方向，后来发现哪里判断错了。
- 时间是怎么被消耗掉的，哪一步最容易拖。
- 队友之间怎么分工，冲突怎么处理。
- 失败之后怎么复盘，下一次提前做什么。

本卷篇目

- 做对了什么，做错了什么：反复验证有效的做法和踩过的坑。
- 团队协作与冲突：分工、沟通、冲突、退出与署名。
- 时间管理与学业平衡：时间预算、优先级、止损。
- 失败复盘：没获奖、没做完、现场翻车之后怎么看。
- 给不同年级的建议：大一、大二、大三分别抓什么。

经验卷只保留可迁移的判断方式，不提供可直接照搬的项目方案。

做对了什么，做错了什么

适用场景

你想知道别人踩过哪些坑，少走一点弯路；或者你刚做完一个项目，想对照着复盘。

做对了什么

1. 先做能演示的最小闭环

不管是机械、电控还是软件，先让“输入—处理—输出”跑通，再谈性能、外观、扩展。很多项目不是输在想法，是输在截止前没有能演示的东西。

2. 把交付物写死

每次分工都写到“文件/演示/测试记录”，不写“负责某模块”。交付物写死，才知道谁卡住、卡在哪。

3. 提前读评分表

评分表不是提交后才看的。先抄下来，逐项找证据，缺什么补什么。比“我觉得做得好”有用。

4. 保留失败证据

失败的照片、日志、测试数据，答辩时主动说。评委更愿意相信一个知道边界的人，而不是只讲成功的人。

5. 文档和代码留痕

决策记录、周报、提交历史，平时觉得麻烦，复盘/答辩/交接时救命。

做错了什么

1. 题目太大

一开始想做“平台”，结果连一个场景都没做透。后来才明白：题目越小，越容易做完；做完再扩。

2. 技术堆砌

为了“看起来厉害”，堆了一堆技术，但没有一个能验收。评委问“具体解决了什么”，答不上来。

3. 分工靠默契

以为大家关系好就不用写清楚，结果有人做得多、有人消失，最后署名/进度都尴尬。

4. 忽视格式和截止

材料内容不错，但页数超了/命名不对/上传太晚，直接出局。规则不是建议，是门槛。

5. 不记录过程

测试数据只有截图，环境/版本/命令都没记。过两周自己都复现不了，答辩被问细节就慌。

6. 用熬夜补进度

短期能顶，长期效率越来越低，错误越来越多。时间预算没算清楚，最后所有事都挤在一起。

如果重来一次

1. 题目压缩到一句话，先做最小闭环。
2. 组队第一天写清分工、交付物、退出机制。
3. 先把评分表变成检查清单。
4. 每周留一次复盘，不等到最后。
5. 所有测试记录环境、版本、命令。
6. 给关键节点留 20% 缓冲。

自检清单

- [ ] 我能说出项目里做对的 3 件事。
- [ ] 我能说出做错的 3 件事。
- [ ] 每条错误有“下次怎么改”。
- [ ] 我保留了失败证据。
- [ ] 我知道下一个项目先做什么。

版本与时效

本文是个人经验，不保证适用于所有赛道。竞赛规则/评审口味每年变化，方法可参考，方向要自己判断。

团队协作与冲突

适用场景

你要组队做竞赛/项目；队友之间出现分歧；有人不干活；有人中途退出；署名谈不拢。

我见过的有效做法

1. 先小人后君子

组队第一天就把丑话说清：

- 谁负责什么，交付物是什么。
- 每周投入多少小时。
- 有人退出怎么交接。
- 署名按什么排。

写成一页纸，大家确认。不是不信任，是避免后面靠猜。

2. 分工到“可验收”

“你负责文档”不如“你负责申报书第 1-5 节，11 月 10 日前交 v1，包含 3 张图”。可验收，才知道有没有完成。

3. 每周一次短会

30 分钟，三个问题：

- 上周完成了什么？
- 这周做什么？
- 有什么阻塞？

不用开长会，但要固定。很多问题不是难，是没人提。

4. 重要决定留文字

群里说“那就这么定了”，过两周没人记得。把决定、理由、待办写进文档/issue，谁都能回看。

5. 冲突先对事实

队友争执时，先对：

- 交付物在哪？
- 什么时候交的？
- 谁在用？
- 证据是什么？

对完事实，很多情绪会降下来。对完还解决不了，再找老师/学长。

我踩过的坑

1. 靠关系组队

关系好不等于能干活。没有分工和退出机制，最后一个人扛，其他人不好意思说，也不敢退出。

2. 队长包办

队长什么都做，其他人插不上手，最后队长累垮，队友也没成长。队长应该做的是定节奏、清阻塞，不是抢所有活。

3. 不说不满

有人对分工不满，但不说，最后用“消失”表达。定期一对一聊 5 分钟，比事后补救有用。

4. 署名最后谈

项目做完才谈署名，谁都觉得自己的贡献最大。应该在中期就确认顺序，按交付物/角色，不按感觉。

5. 退出没有交接

有人退出，代码/文档在他电脑里，其他人接不上。退出机制要写清：交接什么、交给谁、什么时候。

如果重来一次

1. 组队先写一页约定：分工、投入、退出、署名。
2. 每周短会 + 周报，阻塞不过夜。
3. 重要决定留文字。
4. 中期确认署名顺序。
5. 队长做“清障”，不做“包办”。

自检清单

- [ ] 我们有一页组队约定。
- [ ] 分工写到交付物和截止时间。
- [ ] 每周有短会/周报。
- [ ] 重要决定有文字记录。
- [ ] 中期确认过署名顺序。
- [ ] 退出机制写清。
- [ ] 有人负责对外沟通。

版本与时效

本文是个人经验，团队情况各异。涉及学校/赛事规则时，以官方规定为准。

时间管理与学业平衡

适用场景

你在竞赛/项目/课程/英语之间来回切换；总觉得时间不够；期末和比赛撞车；不知道要不要退出某个项目。

我试过有效的做法

1. 先算“可支配时间”

一周 168 小时，扣掉睡眠、上课、吃饭、通勤，剩下才是可支配的。算完就知道“再报一个比赛”是否现实。

2. 按“不可逆性”排优先级

- 考试/绩点：影响保研/毕业，优先。
- 关键竞赛节点：错过等一年。
- 科研/论文：长周期，持续投入。
- 一般活动：可替代，做减法。

不是所有比赛都值得牺牲绩点。

3. 每学期只留 1-2 个主线项目

同时推进 3 个以上项目，通常每个都只做一半。主线之外的项目，要么不做，要么只做明确的小模块。

4. 给截止时间留缓冲

官方截止前 3 天作为内部截止。打印、上传、系统崩溃、队友迟到，都需要缓冲。

5. 设止损线

- 连续两周无实质进展 → 检查方向/分工。
- 关键模块到某日期仍未跑通 → 启用保底方案。
- 时间超预算 30% → 重新评估是否退出。
- 课程出现挂科风险 → 优先保课程。

我踩过的坑

1. 什么都想抓

报了很多比赛，每个都只做了一点。最后没有一个是完整交付的。

2. 用熬夜补时间

短期能顶，第二天效率更低，错误更多。长期看，熬夜是最贵的“省时间”。

3. 不拒绝

队友/老师加任务就接，最后自己的主线被挤掉。学会说“我这周排满了，下周三可以”。

4. 期末前还在改原型

没有提前看考试时间，赛前和期末撞车，两边都做不好。

5. 没有缓冲

把所有时间排满，任何一个意外都会连锁反应。

如果重来一次

1. 每学期初算时间预算，只留 1-2 个主线。
2. 所有截止时间放进一个日历，按周看。
3. 每周留半天缓冲。
4. 给每个项目设止损线。
5. 课程风险出现时，优先保课程。

自检清单

- [ ] 我算过每周可支配时间。
- [ ] 我知道当前主线项目是什么。
- [ ] 我给项目设了止损线。
- [ ] 期末/赛前冲突有提前安排。
- [ ] 每周有缓冲时间。
- [ ] 我没有同时推进超过 2 个主线。
- [ ] 课程出现风险时，我会优先保课程。

版本与时效

本文是个人经验，时间安排因人而异。保研/评奖政策以学校当年文件为准。

失败复盘

适用场景

你参加了一个比赛但没获奖；项目做到一半停了；答辩现场翻车；队友退出。你想知道这些经历到底有什么用。

我经历过的几类失败

1. 没获奖

原因通常不是“评委不懂”，而是：

- 题目太大，没做完。
- 有方案，没证据。
- 材料格式/提交出问题。
- 答辩讲不清重点。

没获奖不等于白做。把交付物、测试记录、复盘留下来，下一个项目能直接用。

2. 做到一半停了

常见原因：

- 关键模块卡住，没有备选方案。
- 队友退出，没人接手。
- 时间被课程/考试挤掉。
- 方向本身不成立。

停下来不一定是错，但要写清“为什么停、停在哪、还有什么能用”。

3. 现场翻车

演示失败、PPT 打不开、设备没电、网络断了。后来做的改变：

- 演示准备本地文件 + 录屏 + 备用设备。
- 提前去场地试。
- 准备“失败话术”，卡住就切录屏。
- 关键数据背下来，不依赖屏幕。

4. 队友/署名纠纷

经历过一次就明白：组队第一天要把分工、退出、署名写清。不是不信任，是保护所有人。

失败后怎么复盘

1. 先写事实

- 目标是什么？
- 实际做到什么？
- 哪个节点开始偏？
- 证据是什么？

不写“大家都很努力”，写交付物和时间线。

2. 找可控原因

把原因分两类：

- **可控**：选题、分工、进度、测试、材料。
- **不可控**：设备故障、政策变化、评委偏好。

复盘重点放在可控项。

3. 提炼一条“下次提前做什么”

不要写 20 条，写 1-3 条能执行的：

- 选题先压到一句话。
- 关键模块准备备选方案。
- 官方截止前 3 天内部截止。
- 答辩前完整试讲两遍。

4. 保留资产

- 代码/图纸/测试数据。
- 文档/PPT/答辩稿。
- 供应商/加工渠道。
- 复盘文档。

失败的价值

失败不会自动变成经验，复盘才会。

- 它告诉你自己的边界：哪些事能做、哪些事需要别人。
- 它告诉你流程的漏洞：哪个节点最容易出问题。
- 它让你在下一个项目里更快判断“这个方向行不行”。

自检清单

- [ ] 我写清了目标 vs 实际。
- [ ] 我找到了可控原因。
- [ ] 我提炼了 1-3 条“下次提前做什么”。
- [ ] 我保留了可复用资产。
- [ ] 我没有把失败全归因于外部。
- [ ] 我没有因为一次失败否定自己。

版本与时效

本文是个人经验。比赛结果受多种因素影响，复盘是为了下一次做得更好，不是为了给自己下结论。

给不同年级的建议

大一：先打底座，别急着刷奖

该抓的

- 绩点：保研/评奖的硬门槛，越早越稳。
- 制图/CAD/编程基础：后面所有项目都要用。
- 英语：四六级尽早过，别拖到大三。
- 一个完整的小项目：哪怕很简单，走完“立项—交付—复盘”。

可以晚一点

- 大规模报比赛：先看清规则和自己的时间。
- 追求“高级技术”：基础不牢，堆技术只会更乱。
- 进很多组织：选 1-2 个真正想做的。

一句话

大一不是“拿奖年”，是“建立工作方式年”。

大二：开始做减法，找到主线

该抓的

- 1-2 个主线竞赛/项目：做完整，不贪多。
- 一个能拿得出手的交付物：原型/论文/专利/完整代码。
- 团队协作：学会分工、周报、复盘。
- 英语/科研入门：为大三做准备。

可以晚一点

- 同时报 5 个比赛：大概率每个都做一半。
- 为了“加分”参加不感兴趣的项目：很难坚持。
- 过早追求“国奖”：先把省级/国家级做完整。

一句话

大二不是“报最多”，是“做完整”。

大三：看结果，也看方向

该抓的

- 保研/考研/出国的硬指标：绩点、英语、科研/竞赛。
- 1-2 个高质量成果：论文/专利/国家级奖项。
- 方向选择：继续本专业/跨方向/就业，尽早想。
- 作品集/简历：把做过的事整理成能看的东西。

可以晚一点

- 为了“刷奖”参加低价值比赛：时间成本太高。
- 什么都想抓：主线之外做减法。
- 和别人比进度：每个人的节奏不同。

一句话

大三不是“越多越好”，是“越清楚越好”。

通用建议

1. **每学期只留 1-2 个主线项目。**

2. **每个项目都要有交付物和复盘。**
3. **英语尽早过线，别拖。**
4. **组队先写约定，再干活。**
5. **材料/规则提前读，别卡截止。**
6. **身体和睡眠是长期战斗力。**

自检清单

- [ ] 我知道自己现在最该抓的 1-2 件事。
- [ ] 我没有同时推进太多主线。
- [ ] 我有一个能拿得出手的交付物。
- [ ] 我知道下一步的方向（保研/考研/就业）。
- [ ] 我保留了项目复盘和资产。

版本与时效

本文是个人经验，不构成规划建议。保研/评奖/升学政策以学校当年文件为准。

卷七 · 工具箱

这一卷是查阅区，不要求从头读。

- 模板索引：Markdown、Office、代码模板。
- 竞赛名录：北京科技大学 2026-09 版认可竞赛 + 工科生优先清单。
- 软件清单：按用途分类的常用软件与选择原则。
- 网站清单：政策、学术、软件文档、数据、设计。
- 检查清单：提交包、AI 声明、答辩前 24 小时。
- 术语表：竞赛、科研、开源、文档里的常见词。
- FAQ：重复出现的问题。

使用原则

1. 工具服从任务，不要为了用工具而用工具。
2. 优先选能长期维护、能导出通用格式的工具。
3. 涉及账号、密钥、隐私的工具，先看安全说明。
4. 清单不是越长越好，能执行才有用。

模板索引

所有模板都在仓库根目录的 `templates/` 与 `checklists/` 下。Office 模板带可见归属行、文档属性与零宽指纹；使用前请保留归属信息。

Markdown 模板

模板	用途
项目启动单	新项目立项：目标、成功标准、里程碑、角色、风险
周报	每周同步：完成、证据、阻塞、下周计划
复盘	项目结束：目标 vs 结果、做对/做错、可复用资产

Office 模板

模板	用途
申报书骨架	Word：封面、摘要、背景、方案、创新点、可行性、进度、风险、经费
项目进度表	Excel：任务、负责人、截止日期、状态、依赖
答辩骨架	PowerPoint：问题、目标、方案、证据、分工、进度、风险

代码模板

模板	用途
Python 项目骨架	`src/` + `tests/` + `pyproject.toml`，可直接复制

检查清单

清单	使用时机
提交包检查	提交前 24 小时
AI 使用声明	使用 AI 辅助写作或编程时
答辩前 24 小时	答辩前一天

使用提醒

- 模板是骨架，不是标准答案。先把“为什么写这一节”想清楚，再填内容。
- Office 模板的页脚归属行与文档属性不要删除；这是溯源的一部分。
- 代码模板的许可是 MIT；正文模板的许可是 CC BY-NC-SA 4.0。

北京科技大学竞赛名录（2026-09 版）

这份名录根据学校 **2026-09 评奖通知附件**整理，分“科技竞赛”和“学科竞赛”两类。面向工科生，但保留全部条目，方便按专业、时间和级别筛选。

> 名录会随年份调整；保研/评奖认定以学校当年文件为准。不要只按“加分”选赛，先看能力匹配、时间成本和成果复用性。

工科生优先看

下面这些和机械、自动化、计算机、材料、能源等工科方向更相关，可以优先了解：

- 中国国际大学生创新大赛
- “挑战杯”全国大学生课外学术科技作品竞赛 / 创业计划竞赛
- 中国机器人及人工智能大赛
- 中国高校智能机器人创意大赛
- 全国大学生机械创新设计大赛
- 中国大学生机械工程创新创意大赛（含毕业设计、起重机、物流等赛项）
- 中国大学生工程实践与创新能力大赛
- 全国大学生智能汽车竞赛
- 全国大学生电子设计竞赛
- 中国大学生飞行器设计创新大赛
- 全国大学生节能减排社会实践与科技竞赛
- 全国大学生数学建模竞赛
- 全国大学生计算机系统能力培养大赛
- 蓝桥杯全国软件和信息技术专业人才大赛
- 全国周培源大学生力学竞赛
- 全国大学生物理实验竞赛
- 北京市大学生工程设计表达竞赛
- 华北五省（市、自治区）大学生机器人大赛

怎么用这份名录

1. **先看级别**：国际级/国家级/省部级/校级，对应学校认定和附加分口径不同。
2. **再看时间**：避开考试周和主线项目冲刺期。
3. **看参赛对象**：有些赛项限专业、限年级、限团队人数。
4. **看赛项**：同一个大赛不同赛项，认可度和难度可能不同。
5. **做减法**：每学期 1-2 个主线，做完比报多更重要。

科技竞赛

序号	竞赛名称	时间	级别	对象
1	中国国际大学生创新大赛	4-11月	国家级	本科生、研究生、毕业5年内校友
2	“挑战杯”全国大学生课外学术科技作品竞赛	单数年6-11月	国家级	本科生、研究生
3	“挑战杯”中国大学生创业计划竞赛	双数年6-11月	国家级	本科生、研究生
4	亚太机器人世界杯国际邀请赛	5月	国际级	本科生
5	大学生iCAN创新创业大赛	11月	国际级	本科生、研究生
6	中国机器人及人工智能大赛	5-7月	国家级	本科生、研究生
7	“西门子杯”中国智能制造挑战赛	7-8月	国家级	本科生、研究生
8	全国大学生创新创业训练计划年会展示	10-12月	国家级	本科生
9	中国青年志愿服务项目大赛（含中国青年志愿服务公益创业赛）	4-9月	国家级	各级青志协或志愿服务联合会、学校志愿服务团体、机关企事业单位志愿服务团

序号	竞赛名称	时间	级别	对象
				体、社会组织、其他志愿服务团体、本科生、研究生
10	全国大学生电子商务“创新、创意及创业”挑战赛	3-9月	国家级	本科生、研究生
11	华为开发者大赛	10月	国家级	本科生、研究生
12	全国大学生机器人科技创新交流营暨机器人大赛	9月	国家级	本科生、研究生
13	中国高校智能机器人创意大赛	8月	国家级	本科生、研究生
14	全国大学生智能汽车竞赛	8月	国家级	本科生
15	中国大学生方程式汽车大赛	10月	国家级	本科生、研究生
16	中国大学生机械工程创新创意大赛-起重机系列赛	3-11月	国家级	本科生
17	全国大学生机器人大赛Robotac对抗赛	6-8月	国家级	本科生
18	全国大学生机器人大赛Robotac三维数字设计赛	6-8月	国家级	本科生

序号	竞赛名称	时间	级别	对象
19	“北斗杯”全国青少年科技创新大赛	3-5月	国家级	本科生
20	全国大学生信息安全与对抗技术竞赛-智能安全赛	9-11月	国家级	本科生、研究生
21	全国大学生节能减排社会实践与科技竞赛	8月	国家级	本科生、研究生
22	中国大学生计算机博弈大赛	8-10月	国家级	本科生、研究生
23	华为ICT大赛	当年10月-次年8月	国家级	本科生
24	中国大学生飞行器设计创新大赛	9月	国家级	本科生
25	全国大学生机器人大赛Robocon系列赛（包含亚太大学生机器人大赛、马术、仿生足式机器人、机器人排球等挑战赛）	6月	国家级	本科生
26	全国大学生机器人大赛RoboMaster机甲大师系列赛（包含高校联盟赛和挑战对抗赛）	8月	国家级	本科生、研究生
27	中国国际飞行器设计挑战赛总决赛暨科研类航空航天模型锦标赛	10月	国家级	本科生

序号	竞赛名称	时间	级别	对象
28	中国大学生工程实践与创新能力大赛	当年10月-次年5月	国家级	本科生
29	中国机器人大赛暨RoboCup机器人世界杯中国赛	4-10月	国家级	本科生、研究生
30	中国研究生创新实践系列大赛	全年	国家级	研究生
31	“锦囊”-未来战争场景与创意大赛	6-12月	国家级	本科生、研究生
32	睿抗机器人开发者大赛（RAICOM）	9月	国家级	本科生、研究生
33	“京彩大创”北京大学生创新创业大赛	4-7月	省部级	本科生、研究生
34	数字汽车大赛	3-11月	省部级	本科生、研究生
35	北京市大学生节能节水低碳减排社会实践与科技竞赛	5月	省部级	本科生
36	北京市卓越联盟工业智能竞赛	6-9月	省部级	本科生

序号	竞赛名称	时间	级别	对象
37	华北五省（市、自治区）大学生机器人大赛	10月	省部级	本科生
38	北京科技大学“摇篮杯”大学生创新创业竞赛（含创意竞赛、科技竞赛、创业竞赛、青年红色筑梦之旅竞赛、华为产业命题竞赛等）	当年9月-次年5月	校级	本科生、研究生、毕业5年内校友

学科竞赛

序号	竞赛名称	时间	级别	对象
1	ASC世界大学生超级计算机竞赛	当年11月-次年3月	国际级	本科生
2	ICPC国际大学生程序设计竞赛	当年5月-次年1月	国际级	本科生
3	全国大学生结构设计竞赛	10-11月	国家级	土木工程专业本科生
4	全国大学生混凝土材料设计大赛	双数年4-8月	国家级	无机非金属材料工程专业、土木工程专业本科生
5	全国大学生岩土工程竞赛	单数年4-7月	国家级	土木工程专业本科生、研究生
6	全国高校安全科学与工程大学生实践与创新作品大赛	9-12月	国家级	本科生
7	全国高等学校采矿工程专业学生实践作品大赛	7-10月	国家级	采矿工程专业本科生
8	全国高等学校矿物加工工程专业学生实践作品大赛	8月	国家级	本科生
9	全国大学生冶金科技竞赛	6-7月	国家级	本科生、研究生

序号	竞赛名称	时间	级别	对象
10	全国模拟炼钢-轧钢大赛	10月	国家级	本科生、研究生
11	全国大学生金相技能大赛	7-8月	国家级	本科生
12	全国大学生机械创新设计大赛	双数年7月	国家级	本科生
13	“高教杯”全国大学生先进成图技术与产品信息建模创新大赛	9月	国家级	本科生
14	全国大学生物流设计大赛	当年11月-次年4月	国家级	物流工程专业、经管类专业本科生
15	中国大学生机械工程创新创意大赛-毕业设计大赛（精雕杯）	3-11月	国家级	本科生
16	全国大学生工业设计大赛	7月	国家级	本科生
17	两岸新锐设计竞赛·华灿奖	1-2月	国家级	工业设计、视觉传达设计专业
18	全国大学生广告艺术大赛	3-6月	国家级	工业设计、视觉传达设计专业
19	未来设计师·全国高校数字艺术设计大赛	6-8月	国家级	工业设计、视觉传达设计专业

序号	竞赛名称	时间	级别	对象
20	中国制冷空调行业大学生科技竞赛	5月	国家级	本科生
21	“华为杯”中国大学生智能设计竞赛	8月	国家级	本科生
22	“台达杯”高校自动化设计大赛	1-7月	国家级	本科生
23	英特尔杯大学生电子设计竞赛嵌入式系统专题邀请赛	3-7月	国家级	本科生
24	全国大学生信息安全竞赛	3-7月	国家级	本科生
25	全国大学生计算机系统能力培养大赛	5-9月	国家级	本科生
26	蓝桥杯全国软件和信息技术专业人才大赛	4-6月	国家级	本科生、研究生
27	中国大学生计算机设计大赛	4-8月	国家级	本科生
28	“中国软件杯”大学生软件设计大赛	6月	国家级	本科生
29	“网鼎杯”网络安全大赛	9-11月	国家级	本科生

序号	竞赛名称	时间	级别	对象
30	中国高校计算机大赛	4-11月	国家级	本科生
31	“大唐杯”全国大学生新一代信息通信技术大赛	3-7月	国家级	本科生、研究生
32	全国大学生数学竞赛	6-11月	国家级	本科生
33	全国大学生数学建模竞赛	9月	国家级	本科生
34	全国大学生统计建模大赛	2-8月	国家级	本科生、研究生
35	全国大学生市场调查与分析大赛	11月	国家级	本科生
36	全国大学生化学实验竞赛	双数年7月	国家级	学习过化学实验课程的本科生
37	全国大学生生命科学竞赛	11月	国家级	本科生
38	全国大学生化学实验创新设计大赛	7月	国家级	本科生
39	全国大学生英语竞赛	4月	国家级	本科生

序号	竞赛名称	时间	级别	对象
40	“21世纪杯”全国英语演讲比赛	4月	国家级	本科生
41	“外研社杯”全国英语演讲、写作、阅读大赛	12月	国家级	本科生
42	全国高校德语专业大学生辩论赛	10-11月	国家级	德语专业本科生
43	“外研社国才杯”国际传播力短视频大赛	3-7月	国家级	本科生
44	“外研社·国才杯”“理解当代中国”全国大学生外语能力大赛相关赛事	5-12月	国家级	英语专业本科生
45	“外研社·国才杯”英语辩论赛	4月	国家级	英语专业本科生
46	中国大学生英语辩论赛	12月	国家级	英语专业本科生
47	“外教社词达人杯”全国大学生英语词汇能力大赛	3-6月	国家级	本科生、研究生
48	全国大学生电子设计竞赛	单数年8月	国家级	本科生
49	全国大学生物理实验竞赛（创新）	9-11月	国家级	理工类各专业本科生

序号	竞赛名称	时间	级别	对象
50	中国大学生物理学术竞赛	7-8月	国家级	理工类各专业本科生
51	全国周培源大学生力学竞赛	单数年	国家级	本科生
52	全国大学生物理实验竞赛（教学）	单数年7月	国家级	理工类各专业本科生
53	“学创杯”全国大学生创业综合模拟大赛	11月	国家级	本科生、研究生
54	中华经典诵写讲大赛	4-9月	国家级	本科生、研究生
55	全国大学生职业规划大赛	当年9月-次年5月	国家级	本科生、研究生
56	全国大学生思政课微课暨红色资源开发成果共享展示活动	6-9月	国家级	本科生
57	全国高校大学生讲思政课公开课	6-9月	国家级	本科生
58	北京市大学生建筑结构设计大赛	9月	省部级	土木工程专业本科生
59	北京市大学生工程设计表达竞赛	5月	省部级	机械专业类本科生

序号	竞赛名称	时间	级别	对象
60	北京市大学生物流设计大赛	9-11月	省部级	物流工程专业、经管类专业本科生
61	北京市大学生动漫设计大赛	7-11月	省部级	本科生
62	北京市大学生文创设计大赛	10月	省部级	本科生
63	华北五省及港澳台大学生计算机应用大赛	9-11月	省部级	本科生
64	全国部分地区大学生物理竞赛	10-12月	省部级	本科生
65	北京市大学生化学实验竞赛	11月	省部级	学习过化学实验课程的本科生
66	北京市大学生生命科学竞赛	3-5月	省部级	本科生
67	北京市大学生化工原理竞赛	4-6月	省部级	化学相关专业学生
68	北京市大学生人文知识竞赛	10月	省部级	本科生
69	北京市大学生模拟法庭竞赛	3-4月	省部级	法学专业本科生

序号	竞赛名称	时间	级别	对象
70	首都大学生诚信演讲比赛	6月	省部级	本科生、研究生
71	北京高校学生跨文化能力大赛	4-10月	省部级	本科生
72	“外教社杯”北京高校青年人才国际胜任力大赛	3-6月	省部级	本科生
73	北京市大学生英语演讲比赛	11月	省部级	本科生
74	大学生“用英语讲好中国故事”优秀短视频作品征集活动	1-6月	省部级	本科生、研究生
75	北京市大学生电子设计竞赛	双数年8月	省部级	本科生
76	北京市大学生集成电路设计大赛	5-6月	省部级	本科生
77	北京市大学生物理学术竞赛	5月	省部级	理工类各专业本科生
78	北京市大学生物理实验竞赛	8-9月	省部级	理工类各专业本科生
79	北京市校际百科知识竞赛	11月	省部级	本科生、研究生

序号	竞赛名称	时间	级别	对象
80	北京高校思想政治理论课学生社会实践优秀论文评选	6-10月	省部级	本科生
81	北京科技大学安全创意作品大赛	5-6月	校级	本科生
82	北京科技大学资源加工与清洁利用大赛	7月	校级	本科生
83	北京科技大学钢铁模拟冶炼大赛	11月	校级	本科生
84	北京科技大学废物利用设计大赛	4月	校级	本科生
85	北京科技大学传感器创新大赛	9-12月	校级	本科生
86	北京科技大学单片机应用大赛	5月	校级	自动化学院、计算机与通信工程学院各专业本科生、对单片机应用技术感兴趣的其他专业本科生
87	北京科技大学化学实验竞赛	4-5月	校级	应用化学专业三年级本科生
88	北京科技大学校园公益营销大赛	4-12月	校级	本科生、研究生
89	北京科技大学外文电影配音大赛	5月	校级	本科生、研究生
90	北京科技大学新生英语朗诵比赛	9月	校级	本科生
91	北京科技大学机器人新媒体创意赛	10月	校级	本科生、研究生

说明

- 本页是整理版，不是学校官方文件；正式申报时以学校/学院当年通知为准。
- 竞赛时间、级别、参赛对象可能调整；报名前再核对一次。
- 名录外的竞赛不一定不能参加，但认定和加分要单独确认。

软件清单

选择原则

1. 工具服从任务，不要为了用工具而用工具。
2. 优先选能导出通用格式、能长期维护的工具。
3. 学校有正版授权的，优先用正版。
4. 破解版/来路不明的“绿色版”不碰：安全、合规、稳定性都不划算。
5. 同一类工具只留 1-2 个，减少切换成本。

文档与写作

用途	工具	备注
长文档/论文	Microsoft Word、WPS	样式、题注、交叉引用
表格/数据	Excel、WPS 表格、LibreOffice Calc	透视表、图表
演示	PowerPoint、WPS 演示	答辩、汇报
Markdown	VS Code、Typora、Obsidian	代码/文档/笔记
PDF	Adobe Acrobat、SumatraPDF、PDF24	阅读、合并、导出
公式	Word 公式、MathType、LaTeX	按学校要求
参考文献	Zotero、EndNote、NoteExpress、Mendeley	文献管理

编程与数据

用途	工具	备注
Python	Python、conda、uv、PyCharm、VS Code	虚拟环境
C/C++	VS Code、Visual Studio、CLion	嵌入式/算法
MATLAB	MATLAB、Octave	仿真/矩阵
版本控制	Git、GitHub Desktop、SourceTree	分支/PR
数据	pandas、NumPy、Excel	清洗/统计
可视化	Matplotlib、Origin、MATLAB	论文图
机器学习	scikit-learn、PyTorch、OpenCV	按项目

机械与仿真

用途	工具	备注
CAD	SolidWorks、CATIA、Inventor、AutoCAD、中望CAD	建模/出图
CAE	ANSYS Workbench、Abaqus、Adams、Isight	结构/运动/优化
电磁/电路	嘉立创EDA、Altium Designer	原理图/PCB
3D 打印	切片软件（Cura/PrusaSlicer/Bambu Studio）	导出 STL
逆向/扫描	SHINING 3D、Geomagic	点云/网格

嵌入式与电子

用途	工具	备注
STM32	STM32CubeMX、Keil、STM32CubeIDE	配置/编译/调试
烧录/串口	ST-Link Utility、VOFA+、串口助手	下载/看数据
网络	Wireshark、PRONETA	抓包/工业网络
逻辑分析	Saleae Logic、示波器软件	调试时序

图示与视觉

用途	工具	备注
流程图	Mermaid、draw.io、Visio	文本/拖拽
矢量图	Adobe Illustrator、Inkscape	海报/示意图
图片	Photoshop、GIMP、ImageMagick	处理/批量
视频	Premiere、DaVinci Resolve、剪映	演示/宣传
3D	Blender、Unity、Unreal	渲染/交互

效率与安全

用途	工具	备注
搜索	Everything、ripgrep	文件名/内容
压缩	7-Zip、Bandizip	恢复记录/加密
密码	Bitwarden、KeePassXC	2FA
笔记	Obsidian、Logseq、OneNote	双链/整理
截图	Snipaste、ShareX	标注/录屏
远程	Windows 远程桌面、向日葵、ToDesk	按需

自检清单

- [ ] 每类工具只留 1-2 个。
- [ ] 学校有正版授权的优先用正版。
- [ ] 没有用来路不明的破解版。
- [ ] 能导出通用格式。
- [ ] 重要数据有备份。

版本与时效

软件清单会过时。安装前看官网当前版本和许可；学校正版授权以信息化办公室/学院通知为准。

网站清单

学校与政策

- 学校官网/教务处/学院官网：培养方案、竞赛认定、保研政策。
- 学校信息化办公室：正版软件、VPN、邮箱。
- 学院/团委通知：竞赛报名、材料提交、公示。
- 图书馆：数据库、查收查引、文献传递。

原则：政策以“当年官方文件”为准，不要用二手整理版。

学术检索

网站	用途
Google Scholar	跨库检索、引文追踪
Web of Science	SCI 检索、引用分析
Scopus	跨学科检索
CNKI	中文期刊/学位论文
arXiv	预印本
PubMed	生物医学
IEEE Xplore	电子/计算机
ScienceDirect	Elsevier 期刊
SpringerLink	Springer 期刊
DOI.org	通过 DOI 找原文

软件与开发

- 官方文档：Python、Git、GitHub、VS Code、MATLAB、SolidWorks 等。
- GitHub：仓库、Issue、Release、Actions。
- Stack Overflow：具体报错。
- MDN：Web 前端。
- Read the Docs：开源项目文档。
- 菜鸟教程/廖雪峰：入门（注意时效）。

数据与统计

- 国家统计局：宏观数据。
- 世界银行/OCED：国际数据。
- 万得/Wind（学校账号）：金融数据。
- Kaggle/UCI：公开数据集。
- 国家地球系统科学数据中心：地理/遥感。
- 中国裁判文书网/专利检索：法律/专利。

设计与素材

- Unsplash/Pexels：免费图片（看许可）。
- Iconfont/Iconify：图标。
- ColorBrewer：配色。
- diagrams.net：在线画图。
- Mermaid Live Editor：在线渲染。

效率与工具

- TinyPNG：图片压缩。
- PDF24：PDF 合并/压缩。
- DeepL/彩云小译：翻译（核对原文）。
- Overleaf：在线 LaTeX。
- Zotero Connector：网页文献抓取。

使用原则

1. 先找官方/一手来源，再看二手整理。
2. 看发布日期/适用版本。
3. 关键结论交叉验证两个独立来源。
4. 不用来路不明的“内部资料/付费保过”。
5. 保存链接、标题、日期，写材料时能回溯。

自检清单

- [ ] 政策类信息来自学校/学院官网。
- [ ] 学术检索用过至少两个数据库。
- [ ] 软件问题先看官方文档。
- [ ] 图片/字体/数据确认许可。
- [ ] 关键结论有两个独立来源。
- [ ] 保存了链接和访问日期。

版本与时效

网站会改版/失效。本文只列类别和常用入口，具体链接以当前官网为准。

术语表

竞赛

- **A 类/B 类竞赛**：学校对竞赛的等级划分，影响保研/评奖认定。以当年文件为准。
- **校赛/省赛/国赛**：选拔层级；同名赛事不同层级含金量不同。
- **网评/会评/答辩**：评审形式；网评看材料，会评专家集中评审，答辩现场陈述+问答。
- **揭榜挂帅**：企业/机构出题，参赛队揭榜解题的赛制。
- **大挑/小挑**：挑战杯“课外学术科技作品竞赛”/“创业计划竞赛”的俗称。
- **路演**：短时间向评委/投资人陈述项目。
- **BP**：商业计划书（Business Plan）。
- **查新**：检索现有技术/专利，证明新颖性。

科研

- **DOI**：数字对象标识符，论文/数据的唯一编号。
- **SCI/EI/中文核心**：期刊收录/评价体系。
- **影响因子**：期刊引用指标，不能直接代表论文质量。
- **同行评审**：审稿人评审。
- **预印本**：未正式发表的论文版本（arXiv 等）。
- **消融实验**：逐个去掉模块，看各自贡献。
- **基线**：对比的基准方法。
- **复现**：别人按你的代码/数据/命令能跑出同样结果。
- **随机种子**：控制随机过程的可复现参数。

开源

- **License**：使用/修改/分发条款。
- **MIT/Apache-2.0**：宽松许可，允许商用。
- **GPL**：传染性许可，衍生作品需同样开源。
- **CC BY-NC-SA**：署名-非商业-相同方式共享。
- **Fork**：复制他人仓库到自己账号。
- **PR**：Pull Request，请求合并修改。
- **Issue**：问题/任务跟踪。
- **Release**：发布版本，附安装包/说明。
- **CI**：持续集成，自动跑测试/构建。

文档与项目

- **里程碑**：可验收的阶段节点。
- **交付物**：可检查的文件/演示/测试结果。
- **干系人**：受影响或能影响项目的人。
- **范围蔓延**：项目不断加需求，导致做不完。
- **止损**：在投入过大前停止/调整。
- **复盘**：项目结束后回顾目标、结果、原因、改进。
- **SOP**：标准操作流程。
- **BOM**：物料清单（Bill of Materials）。

自检清单

- [ ] 遇到不熟的词，先查官方定义。
- [ ] 不用俗称代替正式名称（写材料时）。
- [ ] 术语首次出现时写全称+缩写。
- [ ] 不同赛事同名术语含义可能不同。

版本与时效

术语含义会随政策/社区变化。正式材料以官方文件/标准为准。

FAQ

这本手册是什么

是竞赛题库吗？ 不是。它讲方法、工具、流程和边界，不提供可直接提交的项目方案。

为什么不给具体方向？ 方向每年都变，照搬别人的方案既容易过时，也可能涉及未公开内容。手册给判断框架，方向自己选。

软件版本过时了怎么办？ 每篇末尾有“版本与时效”。优先看官方文档；发现过时可在 issue 里标“章节+日期+变化点”。

竞赛

比赛越多越好吗？ 不是。每学期 1-2 个主线，做完整，比报 5 个各做一半更有用。

校级奖有用吗？ 看学校认定文件。保研/评奖只认名录内赛事和规定级别，以当年文件为准。

没获奖是不是白做？ 不是。交付物、测试记录、复盘都能复用到下一个项目；失败经验本身也有价值。

队友不干活怎么办？ 先对交付物和时间线，再谈分工调整；组队时就应该写清退出机制。

工具

一定要用 Git 吗？ 代码/文档建议用。只写 Word 且不协作，可以先用“日期+版本”命名，但迟早会遇到版本/协作问题。

Typora 收费吗？ Typora 是付费软件，有试用期。预算有限可以用 VS Code + Markdown 插件。

Python 用 conda 还是 venv？ 纯 Python 项目用 venv/uv 就够；需要 C/Fortran 依赖/多版本科学计算，用 conda。

Excel 还是 Python？ 几万行以内、以展示为主，Excel 够用；数据大/需要复现/自动化，用 Python。

开源与安全

私有仓库转公开要注意什么？ 先做隐私/许可/Git 历史检查；删除当前文件不等于删除历史。

别人搬运我的内容怎么办？ 保留证据，联系平台；如果对方去掉署名/商用，按许可条款处理。

AI 写的代码能直接用吗？ 不能。要运行、解释、测试、复现；核心内容按竞赛/课程规则声明。

怎么证明我是原作者？ 可见归属行 + 零宽指纹 + 哈希清单 + 签名提交/tag + 官方地址。

手册使用

可以转载吗？ 正文 CC BY-NC-SA 4.0：署名、非商业、相同方式共享；代码/模板 MIT。

可以用于社团培训吗？ 非商业分享可以，需署名并附仓库链接；商业培训需书面授权。

发现错误怎么反馈？ 开 issue，写清文件、章节、问题、建议；安全问题走私密报告。

自检清单

- [ ] 我先查了目录/搜索，再提问。
- [ ] 我提供了文件/章节/版本。
- [ ] 我没有在公开 issue 里贴隐私信息。
- [ ] 我知道许可边界。

版本与时效

FAQ 会随手册更新。以仓库当前版本为准。

卷八 · 附录

版本

当前版本：`0.1.0`。版本记录见仓库根目录的 CHANGELOG.md。

许可与引用

- 正文：CC BY-NC-SA 4.0。
- 代码与模板：MIT。
- 转载必须署名 `shingja` 并附仓库链接；禁止商业使用。
- 完整说明见许可与引用，法律文本见 NOTICE.md。

唯一官方地址

https://github.com/shingja/ustb-engineering-handbook-shingja

联系

平台	联系方式
GitHub	@shingja
Bilibili	月亮的森林岛
抖音	Sh1ngjA
QQ	`793811299`
邮箱	gy79790920@126.com

问题请开 GitHub issue；安全问题见 SECURITY.md。

许可与引用

许可边界

范围	许可	允许	不允许
`docs/` 正文、图片、站点内容	CC BY-NC-SA 4.0	非商业转载、改编、署名	商业使用、去掉署名
`templates/`、`tools/`、`tests/`、代码片段	MIT	使用、修改、分发、商用	去掉版权声明
第三方软件/字体/竞赛材料	原权利人	按各自许可	未经授权复制/再分发

不确定时：先看文件头/目录 README；没有明确许可的，不要直接用。

引用格式

学术/报告引用：

```text
shingja. shingja 的工科生手册[EB/OL]. 2026.
https://github.com/shingja/ustb-engineering-handbook-shingja
```

网页/文档引用：

```text
来源：shingja 的工科生手册
https://github.com/shingja/ustb-engineering-handbook-shingja
许可：CC BY-NC-SA 4.0
```

转载要求：

1. 标明作者 `shingja` 和仓库链接。
2. 保留许可与 NOTICE。
3. 不得移除文件内的归属行。
4. 改编作品以相同许可共享。
5. 不得声称原创或暗示作者背书。

商业使用

以下情况需事先书面授权：

- 付费课程/培训/社群。
- 付费资料/会员内容。
- 商业咨询/外包交付。
- 以本手册为主要卖点的产品。
- 任何以营利为目的的分发。

联系：GitHub @shingja。

第三方素材

- 软件截图：版权归软件厂商，引用时注明。
- 字体：确认商用/再分发许可。
- 竞赛官方材料：版权归主办方，不复制原件。
- 论文/图表：按出版方许可引用。
- 数据集：按数据集许可/隐私要求使用。

引用与 AI

- 用本手册训练/生成内容，需遵守仓库 `robots.txt` / `ai.txt` 与许可。
- AI 引用本手册时，应保留作者、链接、许可。
- 不得用 AI 批量洗稿后声称原创。

自检清单

- [ ] 我确认了要用的内容属于哪个许可。
- [ ] 转载保留了署名/链接/许可。
- [ ] 没有移除归属行/指纹。
- [ ] 商业使用已取得授权。
- [ ] 第三方素材有来源和许可。
- [ ] 引用格式完整。

版本与时效

许可文本以 `LICENSE` / `LICENSE-CODE` / `NOTICE.md` 为准；本页只是说明，不替代法律文本。

Error response

Error code: 404

Message: File not found.

Error code explanation: 404 - Nothing matches the given URI.